HCL CAREER SHAPER PROGRAM

PROJECT TOPIC

HRGenius: HR Management System (HRMS)

Employee data management, recruitment/onboarding, payroll, performance management and HR analytics.

CODE FILES

Ishan Arora
Roll No.: 2300321530091
ishan.23b1531004@abes.ac.in
CSE-AIML
IVTH YEAR
ABES Engineering College

INDEX

01 BACKEND CODE

Java source and test files — 48 files

02 FRONTEND CODE

HTML, JavaScript and CSS — 3 files

03 BUILD & CONFIGURATION FILES

Maven, runtime configuration and API collection — 5 files

01. BACKEND CODE

src/main/java/com/hrgenius/HrGeniusApplication.java

```
0001 package com.hrgenius;
0002 import org.springframework.boot.SpringApplication;
0003 import org.springframework.boot.autoconfigure.SpringBootApplication;
0004 @SpringBootApplication
0005 public class HrGeniusApplication {
0006  public static void main(String[] args) { SpringApplication.run(HrGeniusApplication.class, args); }
0007 }
```

src/main/java/com/hrgenius/config/DataSeeder.java

```
0001 package com.hrgenius.config;
0002
0003 import com.hrgenius.entity.*;
0004 import com.hrgenius.repository.*;
0005 import org.springframework.boot.CommandLineRunner;
0006 import org.springframework.context.annotation.Bean;
0007 import org.springframework.context.annotation.Configuration;
0008 import org.springframework.security.crypto.password.PasswordEncoder;
0009
0010 import java.math.BigDecimal;
0011 import java.time.LocalDate;
0012 import java.time.LocalDateTime;
0013
0014 @Configuration
0015 public class DataSeeder {
0016  @Bean
0017  CommandLineRunner seed(
0018      UserRepository users,
0019      DepartmentRepository departments,
0020      EmployeeRepository employees,
0021      JobOpeningRepository jobs,
0022      CandidateRepository candidates,
0023      InterviewRepository interviews,
0024      SalaryStructureRepository salaryStructures,
0025      PerformanceCycleRepository cycles,
0026      GoalRepository goals,
0027      PerformanceReviewRepository reviews,
0028      OnboardingTaskRepository onboardingTasks,
0029      PasswordEncoder encoder
0030  ) {
0031   return args -> {
0032    // Seed Departments
0033    Department hrDept;
0034    Department engDept;
0035    if (departments.count() == 0) {
0036     hrDept = new Department();
0037     hrDept.setName("Human Resources");
0038     hrDept.setDescription("Core HR operations, recruitment, and employee relations");
0039     hrDept = departments.save(hrDept);
0040
0041     engDept = new Department();
0042     engDept.setName("Engineering");
0043     engDept.setDescription("Software engineering, architecture, and technology infrastructure");
0044     engDept = departments.save(engDept);
0045
0046     Department prodDept = new Department();
```

```
0047     prodDept.setName("Product & Design");
0048     prodDept.setDescription("Product development, UX research, and design systems");
0049     departments.save(prodDept);
0050    } else {
0051     hrDept = departments.findAll().get(0);
0052     engDept = departments.findAll().size() > 1 ? departments.findAll().get(1) : hrDept;
0053    }
0054
0055    // Seed Employees
0056    Employee adminEmp = null;
0057    if (employees.count() == 0) {
0058     adminEmp = new Employee();
0059     adminEmp.setEmployeeCode("EMP-1001");
0060     adminEmp.setName("System Administrator");
0061     adminEmp.setEmail("admin");
0062     adminEmp.setPhone("+91 98765 43210");
0063     adminEmp.setDesignation("HR Operations Lead");
0064     adminEmp.setStatus("ACTIVE");
0065     adminEmp.setDepartmentId(hrDept.getId());
0066     adminEmp.setDateOfJoining(LocalDate.of(2023, 1, 15));
0067     adminEmp = employees.save(adminEmp);
0068
0069     Employee emp2 = new Employee();
0070     emp2.setEmployeeCode("EMP-1002");
0071     emp2.setName("Priya Sharma");
0072     emp2.setEmail("priya.sharma@hrgenius.local");
0073     emp2.setPhone("+91 98111 22334");
0074     emp2.setDesignation("Senior Software Engineer");
0075     emp2.setStatus("ACTIVE");
0076     emp2.setDepartmentId(engDept.getId());
0077     emp2.setDateOfJoining(LocalDate.of(2023, 6, 1));
0078     emp2.setManagerId(adminEmp.getId());
0079     emp2 = employees.save(emp2);
0080
0081     Employee emp3 = new Employee();
0082     emp3.setEmployeeCode("EMP-1003");
0083     emp3.setName("Arun Patel");
0084     emp3.setEmail("arun.patel@hrgenius.local");
0085     emp3.setPhone("+91 98222 33445");
0086     emp3.setDesignation("Product Designer");
0087     emp3.setStatus("ACTIVE");
0088     emp3.setDepartmentId(hrDept.getId());
0089     emp3.setDateOfJoining(LocalDate.of(2024, 2, 10));
0090     emp3 = employees.save(emp3);
0091    } else {
0092     adminEmp = employees.findAll().get(0);
0093    }
0094
0095    // Seed Admin User
0096    if (users.findByUsername("admin").isEmpty()) {
0097     User u = new User();
0098     u.setUsername("admin");
0099     u.setPassword(encoder.encode(System.getenv().getOrDefault("HRGENIUS_ADMIN_PASSWORD", "Admin@12345")));
0100     u.setRole("ADMIN");
0101     u.setStatus("ACTIVE");
0102     if (adminEmp != null) {
0103      u.setEmployeeId(adminEmp.getId());
0104     }
0105     users.save(u);
0106    }
0107
```

```
    // Seed Salary Structures
    if (salaryStructures.count() == 0 && adminEmp != null) {
     SalaryStructure ss = new SalaryStructure();
     ss.setEmployeeId(adminEmp.getId());
     ss.setBasic(new BigDecimal("75000.00"));
     ss.setHra(new BigDecimal("30000.00"));
     ss.setOtherAllowances(new BigDecimal("15000.00"));
     ss.setDeductions(new BigDecimal("9000.00"));
     salaryStructures.save(ss);

     for (Employee emp : employees.findAll()) {
      if (!emp.getId().equals(adminEmp.getId()) && salaryStructures.findByEmployeeId(emp.getId()).isEmpty()) {
       SalaryStructure s = new SalaryStructure();
       s.setEmployeeId(emp.getId());
       s.setBasic(new BigDecimal("60000.00"));
       s.setHra(new BigDecimal("24000.00"));
       s.setOtherAllowances(new BigDecimal("10000.00"));
       s.setDeductions(new BigDecimal("7200.00"));
       salaryStructures.save(s);
      }
     }
    }

    // Seed Job Openings
    JobOpening job1 = null;
    if (jobs.count() == 0) {
     job1 = new JobOpening();
     job1.setTitle("Senior Full Stack Java Engineer");
     job1.setDepartmentId(engDept.getId());
     job1.setDescription("Leading end-to-end development of microservices using Spring Boot, React, and cloud native architectures.");
     job1.setLocation("Bengaluru / Hybrid");
     job1.setVacancies(3);
     job1.setStatus("OPEN");
     job1.setPostedDate(LocalDate.now().minusDays(12));
     job1 = jobs.save(job1);

     JobOpening job2 = new JobOpening();
     job2.setTitle("Talent Acquisition Specialist");
     job2.setDepartmentId(hrDept.getId());
     job2.setDescription("Manage full life-cycle recruiting for engineering, product, and leadership tracks.");
     job2.setLocation("Noida / On-site");
     job2.setVacancies(2);
     job2.setStatus("OPEN");
     job2.setPostedDate(LocalDate.now().minusDays(5));
     jobs.save(job2);
    } else {
     job1 = jobs.findAll().get(0);
    }

    // Seed Candidates & Interviews
    if (candidates.count() == 0 && job1 != null) {
     Candidate c1 = new Candidate();
     c1.setName("Rohan Verma");
     c1.setEmail("rohan.verma@example.com");
     c1.setPhone("+91 99887 76655");
     c1.setJobOpeningId(job1.getId());
     c1.setApplicationStatus("SHORTLISTED");
     c1.setAppliedAt(LocalDate.now().minusDays(6));
     c1 = candidates.save(c1);

     Candidate c2 = new Candidate();
```

```
    c2.setName("Sneha Iyer");
    c2.setEmail("sneha.iyer@example.com");
    c2.setPhone("+91 99112 23344");
    c2.setJobOpeningId(job1.getId());
    c2.setApplicationStatus("APPLIED");
    c2.setAppliedAt(LocalDate.now().minusDays(2));
    candidates.save(c2);

    if (interviews.count() == 0 && adminEmp != null) {
     Interview inv = new Interview();
     inv.setCandidateId(c1.getId());
     inv.setInterviewerId(adminEmp.getId());
     inv.setScheduledDate(LocalDateTime.now().plusDays(2).withHour(14).withMinute(0));
     inv.setMode("Google Meet");
     inv.setMeetingLink("https://meet.google.com/hrg-demo-tech");
     inv.setResult("SCHEDULED");
     interviews.save(inv);
    }
   }

   // Seed Performance Cycles & Goals
   if (cycles.count() == 0) {
    PerformanceCycle pc = new PerformanceCycle();
    pc.setName("Annual Performance Review 2026");
    pc.setStartDate(LocalDate.of(2026, 1, 1));
    pc.setEndDate(LocalDate.of(2026, 12, 31));
    pc.setStatus("ACTIVE");
    pc = cycles.save(pc);

    if (adminEmp != null && goals.count() == 0) {
     Goal g = new Goal();
     g.setEmployeeId(adminEmp.getId());
     g.setCycleId(pc.getId());
     g.setTitle("Automate HR Payroll & Onboarding Pipelines");
     g.setDescription("Streamline compensation runs and automate joining document workflows");
     g.setStatus("IN_PROGRESS");
     g.setDueDate(LocalDate.of(2026, 6, 30));
     goals.save(g);
    }
   }

   // Seed Initial Onboarding Tasks
   if (onboardingTasks.count() == 0 && adminEmp != null) {
    OnboardingTask task = new OnboardingTask();
    task.setEmployeeId(adminEmp.getId());
    task.setTaskName("Review and sign IT policy guidelines");
    task.setDescription("Standard information security and workplace compliance");
    task.setStatus("COMPLETED");
    task.setDueDate(LocalDate.now().minusDays(10));
    onboardingTasks.save(task);
   }
  };
 }
}
```

src/main/java/com/hrgenius/config/SecurityConfig.java

```
package com.hrgenius.config;

import com.hrgenius.security.JwtFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
0006 import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
0007 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
0008 import org.springframework.security.config.http.SessionCreationPolicy;
0009 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
0010 import org.springframework.security.crypto.password.PasswordEncoder;
0011 import org.springframework.security.web.SecurityFilterChain;
0012 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
0013
0014 @Configuration
0015 @EnableMethodSecurity
0016 public class SecurityConfig {
0017
0018     @Bean
0019     public PasswordEncoder passwordEncoder() {
0020         return new BCryptPasswordEncoder();
0021     }
0022
0023     @Bean
0024     public SecurityFilterChain filterChain(HttpSecurity http, JwtFilter jwt) throws Exception {
0025         return http
0026             .csrf(c -> c.disable())
0027             .cors(c -> {})
0028             .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
0029             .authorizeHttpRequests(a -> a
0030                 .requestMatchers(
0031                     "/",
0032                     "/index.html",
0033                     "/app.js",
0034                     "/styles.css",
0035                     "/favicon.ico",
0036                     "/api/v1/auth/**",
0037                     "/api/v1/jobs",
0038                     "/api/v1/jobs/**",
0039                     "/swagger-ui/**",
0040                     "/v3/api-docs/**",
0041                     "/swagger-ui.html"
0042                 ).permitAll()
0043                 .anyRequest().authenticated()
0044             )
0045             .addFilterBefore(jwt, UsernamePasswordAuthenticationFilter.class)
0046             .build();
0047     }
0048 }
```

src/main/java/com/hrgenius/controller/AnalyticsController.java

```
0001 package com.hrgenius.controller;
0002
0003 import com.hrgenius.entity.*;
0004 import com.hrgenius.repository.*;
0005 import org.springframework.security.access.prepost.PreAuthorize;
0006 import org.springframework.web.bind.annotation.*;
0007
0008 import java.math.BigDecimal;
0009 import java.time.Month;
0010 import java.util.*;
0011
0012 @RestController
0013 @RequestMapping("/api/v1/analytics")
0014 @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0015 public class AnalyticsController {
0016  private final EmployeeRepository employeeRepo;
```

```
0017  private final DepartmentRepository deptRepo;
0018  private final CandidateRepository candidateRepo;
0019  private final PayrollRepository payrollRepo;
0020  private final SalaryStructureRepository salaryRepo;
0021  private final PerformanceReviewRepository reviewRepo;
0022
0023  public AnalyticsController(
0024      EmployeeRepository employeeRepo,
0025      DepartmentRepository deptRepo,
0026      CandidateRepository candidateRepo,
0027      PayrollRepository payrollRepo,
0028      SalaryStructureRepository salaryRepo,
0029      PerformanceReviewRepository reviewRepo
0030  ) {
0031   this.employeeRepo = employeeRepo;
0032   this.deptRepo = deptRepo;
0033   this.candidateRepo = candidateRepo;
0034   this.payrollRepo = payrollRepo;
0035   this.salaryRepo = salaryRepo;
0036   this.reviewRepo = reviewRepo;
0037  }
0038
0039  @GetMapping("/headcount")
0040  public Map<String, Object> getHeadcount() {
0041   List<Employee> employees = employeeRepo.findAll();
0042   long activeCount = employees.stream().filter(e -> "ACTIVE".equalsIgnoreCase(e.getStatus())).count();
0043
0044   Map<Long, String> deptNames = new HashMap<>();
0045   for (Department d : deptRepo.findAll()) {
0046    deptNames.put(d.getId(), d.getName());
0047   }
0048
0049   Map<String, Integer> deptCounts = new LinkedHashMap<>();
0050   for (Employee e : employees) {
0051    if ("ACTIVE".equalsIgnoreCase(e.getStatus())) {
0052     String dept = (e.getDepartmentId() != null && deptNames.containsKey(e.getDepartmentId()))
0053         ? deptNames.get(e.getDepartmentId())
0054         : "Unassigned";
0055     deptCounts.put(dept, deptCounts.getOrDefault(dept, 0) + 1);
0056   }
0057   }
0058
0059   List<Map<String, Object>> chart = new ArrayList<>();
0060   if (deptCounts.isEmpty()) {
0061    chart.add(Map.of("label", "General", "value", (int) activeCount));
0062   } else {
0063    deptCounts.forEach((dept, count) -> chart.add(Map.of("label", dept, "value", count)));
0064   }
0065
0066   return Map.of(
0067       "headcount", activeCount,
0068       "total", activeCount,
0069       "data", chart
0070   );
0071  }
0072
0073  @GetMapping("/hiring")
0074  public Map<String, Object> getHiring() {
0075   List<Candidate> candidates = candidateRepo.findAll();
0076   long hires = candidates.stream()
0077       .filter(c -> "SELECTED".equalsIgnoreCase(c.getApplicationStatus()))
```

```
0078        .count();
0079
0080    Map<String, Integer> statusCounts = new LinkedHashMap<>();
0081    statusCounts.put("APPLIED", 0);
0082    statusCounts.put("SHORTLISTED", 0);
0083    statusCounts.put("INTERVIEWED", 0);
0084    statusCounts.put("SELECTED", 0);
0085    statusCounts.put("REJECTED", 0);
0086
0087    for (Candidate c : candidates) {
0088     String s = c.getApplicationStatus() != null ? c.getApplicationStatus().toUpperCase() : "APPLIED";
0089     statusCounts.put(s, statusCounts.getOrDefault(s, 0) + 1);
0090    }
0091
0092    List<Map<String, Object>> chart = new ArrayList<>();
0093    statusCounts.forEach((status, count) -> chart.add(Map.of("label", status, "value", count)));
0094
0095    return Map.of(
0096        "totalHires", hires,
0097        "total", candidates.size(),
0098        "data", chart
0099    );
0100   }
0101
0102   @GetMapping("/attrition")
0103   public Map<String, Object> getAttrition() {
0104    List<Employee> employees = employeeRepo.findAll();
0105    long active = employees.stream().filter(e -> "ACTIVE".equalsIgnoreCase(e.getStatus())).count();
0106    long inactive = employees.stream().filter(e -> "INACTIVE".equalsIgnoreCase(e.getStatus())).count();
0107
0108    List<Map<String, Object>> chart = new ArrayList<>();
0109    chart.add(Map.of("label", "Active Staff", "value", (int) active));
0110    chart.add(Map.of("label", "Inactive / Exited", "value", (int) inactive));
0111
0112    return Map.of(
0113        "totalAttrition", inactive,
0114        "total", inactive,
0115        "data", chart
0116    );
0117   }
0118
0119   @GetMapping("/payroll-cost")
0120   public Map<String, Object> getPayrollCost() {
0121    List<Payroll> payrollList = payrollRepo.findAll();
0122    BigDecimal totalCost = BigDecimal.ZERO;
0123    Map<String, BigDecimal> monthlyTotals = new LinkedHashMap<>();
0124
0125    if (!payrollList.isEmpty()) {
0126     for (Payroll p : payrollList) {
0127      totalCost = totalCost.add(p.getNetPay());
0128      String label = Month.of(Math.max(1, Math.min(12, p.getMonth()))).name().substring(0, 3) + " " + p.getYear();
0129      monthlyTotals.put(label, monthlyTotals.getOrDefault(label, BigDecimal.ZERO).add(p.getNetPay()));
0130     }
0131    } else {
0132     // Compute from user-defined active salary structures
0133     for (SalaryStructure s : salaryRepo.findAll()) {
0134      BigDecimal gross = s.getBasic().add(s.getHra()).add(s.getOtherAllowances());
0135      BigDecimal net = gross.subtract(s.getDeductions());
0136      totalCost = totalCost.add(net);
0137     }
0138     if (totalCost.compareTo(BigDecimal.ZERO) > 0) {
```

```
    monthlyTotals.put("Current Est.", totalCost);
   }
  }

  List<Map<String, Object>> chart = new ArrayList<>();
  monthlyTotals.forEach((period, amt) -> chart.add(Map.of("label", period, "value", amt.intValue())));

  return Map.of(
      "totalCost", "■" + totalCost.setScale(2, java.math.RoundingMode.HALF_UP).toPlainString(),
      "total", totalCost.intValue(),
      "data", chart
  );
 }

 @GetMapping("/performance")
 public Map<String, Object> getPerformance() {
  List<PerformanceReview> reviews = reviewRepo.findAll();
  Map<Integer, Integer> distribution = new LinkedHashMap<>();
  for (int i = 1; i <= 5; i++) distribution.put(i, 0);

  for (PerformanceReview r : reviews) {
   if (r.getRating() != null && r.getRating() >= 1 && r.getRating() <= 5) {
    distribution.put(r.getRating(), distribution.get(r.getRating()) + 1);
   }
  }

  List<Map<String, Object>> chart = new ArrayList<>();
  distribution.forEach((star, count) -> chart.add(Map.of("label", star + " Stars", "value", count)));

  return Map.of(
      "totalReviews", reviews.size(),
      "total", reviews.size(),
      "data", chart
  );
 }
}
```

src/main/java/com/hrgenius/controller/AuthController.java

```
package com.hrgenius.controller;

import com.hrgenius.dto.LoginRequest;
import com.hrgenius.service.AuthService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
import java.util.Map;

@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {
 private final AuthService auth;

 public AuthController(AuthService auth) {
  this.auth = auth;
 }

 @PostMapping("/login")
 public Map<String,Object> login(@Valid @RequestBody LoginRequest r) {
  return auth.login(r.username(), r.password());
 }
```

```
0024
0025  @PostMapping("/candidate-register")
0026  public ResponseEntity<?> register(@RequestBody Map<String,String> body) {
0027   auth.registerCandidate(body.get("username"), body.get("password"));
0028   return ResponseEntity.status(201).body(Map.of("message", "Candidate account created"));
0029  }
0030
0031  @PostMapping("/change-password")
0032  public ResponseEntity<?> changePassword(@RequestBody Map<String,String> body, Authentication authentication) {
0033   if (authentication == null || authentication.getName() == null) {
0034    return ResponseEntity.status(401).body(Map.of("message", "Authentication required"));
0035   }
0036   auth.changePassword(authentication.getName(), body.get("currentPassword"), body.get("newPassword"));
0037   return ResponseEntity.ok(Map.of("message", "Password changed successfully"));
0038  }
0039 }
```

src/main/java/com/hrgenius/controller/CandidateController.java

```
0001 package com.hrgenius.controller;
0002
0003 import com.hrgenius.entity.Candidate;
0004 import com.hrgenius.entity.Department;
0005 import com.hrgenius.entity.Employee;
0006 import com.hrgenius.entity.JobOpening;
0007 import com.hrgenius.entity.OnboardingTask;
0008 import com.hrgenius.repository.CandidateRepository;
0009 import com.hrgenius.repository.DepartmentRepository;
0010 import com.hrgenius.repository.EmployeeRepository;
0011 import com.hrgenius.repository.JobOpeningRepository;
0012 import com.hrgenius.repository.OnboardingTaskRepository;
0013 import org.springframework.http.ResponseEntity;
0014 import org.springframework.security.access.prepost.PreAuthorize;
0015 import org.springframework.security.core.Authentication;
0016 import org.springframework.web.bind.annotation.*;
0017
0018 import java.time.LocalDate;
0019 import java.util.*;
0020
0021 @RestController
0022 @RequestMapping("/api/v1/candidates")
0023 public class CandidateController {
0024  private final CandidateRepository candidateRepo;
0025  private final JobOpeningRepository jobRepo;
0026  private final EmployeeRepository employeeRepo;
0027  private final DepartmentRepository deptRepo;
0028  private final OnboardingTaskRepository onboardingRepo;
0029
0030  public CandidateController(
0031      CandidateRepository candidateRepo,
0032      JobOpeningRepository jobRepo,
0033      EmployeeRepository employeeRepo,
0034      DepartmentRepository deptRepo,
0035      OnboardingTaskRepository onboardingRepo
0036  ) {
0037   this.candidateRepo = candidateRepo;
0038   this.jobRepo = jobRepo;
0039   this.employeeRepo = employeeRepo;
0040   this.deptRepo = deptRepo;
0041   this.onboardingRepo = onboardingRepo;
0042  }
0043
```

```
@GetMapping
@PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
public List<Map<String, Object>> list(
    @RequestParam(required = false) Long jobId,
    @RequestParam(required = false) String status
) {
 Map<Long, String> jobMap = new HashMap<>();
 for (JobOpening j : jobRepo.findAll()) {
  jobMap.put(j.getId(), j.getTitle());
 }

 List<Candidate> candidates;
 if (jobId != null && status != null && !status.isBlank()) {
  candidates = candidateRepo.findByJobOpeningIdAndApplicationStatus(jobId, status);
 } else if (jobId != null) {
  candidates = candidateRepo.findByJobOpeningId(jobId);
 } else if (status != null && !status.isBlank()) {
  candidates = candidateRepo.findByApplicationStatus(status);
 } else {
  candidates = candidateRepo.findAll();
 }

 List<Map<String, Object>> result = new ArrayList<>();
 for (Candidate c : candidates) {
  Map<String, Object> map = new LinkedHashMap<>();
  map.put("id", c.getId());
  map.put("name", c.getName());
  map.put("candidateName", c.getName());
  map.put("email", c.getEmail());
  map.put("phone", c.getPhone() != null ? c.getPhone() : "");
  map.put("jobOpeningId", c.getJobOpeningId());
  map.put("jobTitle", jobMap.getOrDefault(c.getJobOpeningId(), "General Position"));
  map.put("status", c.getApplicationStatus());
  map.put("applicationStatus", c.getApplicationStatus());
  map.put("appliedAt", c.getAppliedAt());
  result.add(map);
 }
 return result;
}

@GetMapping("/me/applications")
public List<Map<String, Object>> myApplications(Authentication auth) {
 String username = auth != null ? auth.getName() : "";
 Map<Long, JobOpening> jobMap = new HashMap<>();
 for (JobOpening j : jobRepo.findAll()) {
  jobMap.put(j.getId(), j);
 }

 List<Candidate> userCandidates = candidateRepo.findAll().stream()
     .filter(c -> c.getEmail().equalsIgnoreCase(username)
         || c.getName().equalsIgnoreCase(username)
         || (username.contains("@") && c.getEmail().equalsIgnoreCase(username)))
     .toList();

 List<Map<String, Object>> result = new ArrayList<>();
 for (Candidate c : userCandidates) {
  JobOpening job = jobMap.get(c.getJobOpeningId());
  Map<String, Object> map = new LinkedHashMap<>();
  map.put("id", c.getId());
  map.put("jobTitle", job != null ? job.getTitle() : "Position");
  map.put("location", job != null ? job.getLocation() : "Hybrid");
```

```
    map.put("status", c.getApplicationStatus());
    map.put("appliedAt", c.getAppliedAt());
    result.add(map);
   }
   return result;
  }

  @PutMapping("/{id}/status")
  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
  public ResponseEntity<?> updateStatus(@PathVariable Long id, @RequestBody Map<String, String> body) {
   Candidate c = candidateRepo.findById(id).orElseThrow(() -> new NoSuchElementException("Candidate not found"));
   String newStatus = body.get("status");
   if (newStatus != null) {
    c.setApplicationStatus(newStatus);
    candidateRepo.save(c);
   }
   return ResponseEntity.ok(Map.of("message", "Status updated successfully", "status", c.getApplicationStatus()));
  }

  @PostMapping("/{id}/convert")
  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
  public ResponseEntity<?> convertToEmployee(@PathVariable Long id, @RequestBody(required = false) Map<String, Object> body) {
   Candidate c = candidateRepo.findById(id).orElseThrow(() -> new NoSuchElementException("Candidate not found"));
   JobOpening job = jobRepo.findById(c.getJobOpeningId()).orElse(null);

   Employee e = new Employee();
   e.setEmployeeCode("EMP-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase());
   e.setName(c.getName());
   e.setEmail(c.getEmail());

   // Phone from user body if provided, else candidate's phone
   String phone = (body != null && body.containsKey("phone") && body.get("phone") != null)
       ? String.valueOf(body.get("phone"))
       : (c.getPhone() != null ? c.getPhone() : "");
   e.setPhone(phone);

   // Designation from user body or job title
   String designation = (body != null && body.containsKey("designation") && body.get("designation") != null)
       ? String.valueOf(body.get("designation"))
       : (job != null ? job.getTitle() : "Associate");
   e.setDesignation(designation);

   // Department from user body or job department or dynamic first department
   Long deptId = null;
   if (body != null && body.containsKey("departmentId") && body.get("departmentId") != null) {
    deptId = ((Number) body.get("departmentId")).longValue();
   } else if (job != null && job.getDepartmentId() != null) {
    deptId = job.getDepartmentId();
   } else {
    deptId = deptRepo.findAll().stream().findFirst().map(Department::getId).orElse(null);
   }
   e.setDepartmentId(deptId);

   e.setDateOfJoining(LocalDate.now());
   e.setStatus("ACTIVE");
   Employee saved = employeeRepo.save(e);

   c.setApplicationStatus("SELECTED");
   candidateRepo.save(c);

   // Generate onboarding checklist for new hire
```

```
0166   List<String> defaultTasks = List.of(
0167       "Complete employee personal profile",
0168       "Upload identity and educational documents",
0169       "Sign HR policy and employment contract",
0170       "Collect IT workstation & configure credentials"
0171   );
0172   for (String taskName : defaultTasks) {
0173    OnboardingTask task = new OnboardingTask();
0174    task.setEmployeeId(saved.getId());
0175    task.setTaskName(taskName);
0176    task.setDescription("Onboarding requirement for " + saved.getName());
0177    task.setStatus("PENDING");
0178    task.setDueDate(LocalDate.now().plusDays(7));
0179    onboardingRepo.save(task);
0180   }
0181
0182   return ResponseEntity.ok(Map.of(
0183       "message", "Candidate successfully converted to Employee",
0184       "employeeId", saved.getId(),
0185       "employeeCode", saved.getEmployeeCode()
0186   ));
0187  }
0188 }
```

src/main/java/com/hrgenius/controller/DepartmentController.java

```
0001 package com.hrgenius.controller;
0002 import com.hrgenius.entity.Department; import com.hrgenius.repository.DepartmentRepository;
0003 import org.springframework.http.*; import org.springframework.security.access.prepost.PreAuthorize; import org.springframework.web.bind.annotation.*;
0004 @RestController @RequestMapping("/api/v1/departments")
0005 public class DepartmentController {
0006  private final DepartmentRepository repo; public DepartmentController(DepartmentRepository repo){this.repo=repo;}
0007  @PostMapping @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')") public ResponseEntity<Department> create(@RequestBody Department d){d.setId(null);return ResponseEntity.status(201).bo
dy(repo.save(d));}
0008  @GetMapping public java.util.List<Department> list(){return repo.findAll();}
0009  @PutMapping("/{id}") @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')") public Department update(@PathVariable long id,@RequestBody Department d){Department x=repo.findById(id).orEls
eThrow();x.setName(d.getName());x.setDescription(d.getDescription());return repo.save(x);}
0010  @DeleteMapping("/{id}") @PreAuthorize("hasRole('ADMIN')") public ResponseEntity<?> delete(@PathVariable long id){repo.deleteById(id);return ResponseEntity.noContent().build();}
0011 }
```

src/main/java/com/hrgenius/controller/EmployeeController.java

```
0001 package com.hrgenius.controller;
0002
0003 import com.hrgenius.dto.*;
0004 import com.hrgenius.service.EmployeeService;
0005 import jakarta.validation.Valid;
0006 import org.springframework.data.domain.*;
0007 import org.springframework.http.*;
0008 import org.springframework.security.access.prepost.PreAuthorize;
0009 import org.springframework.security.core.Authentication;
0010 import org.springframework.web.bind.annotation.*;
0011
0012 import java.util.List;
0013
0014 @RestController
0015 @RequestMapping("/api/v1/employees")
0016 public class EmployeeController {
0017  private final EmployeeService service;
0018
0019  public EmployeeController(EmployeeService service) {
0020   this.service = service;
0021  }
```

```
0022
0023  @PostMapping
0024  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0025  public ResponseEntity<EmployeeResponse> create(@Valid @RequestBody EmployeeRequest r) {
0026   return ResponseEntity.status(201).body(service.getResponse(service.create(r).getId()));
0027  }
0028
0029  @GetMapping
0030  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER','MANAGER')")
0031  public Page<EmployeeResponse> list(@RequestParam(required=false) String search, Pageable pageable) {
0032   return service.list(search, pageable);
0033  }
0034
0035  @GetMapping("/me")
0036  public ResponseEntity<EmployeeResponse> getCurrentEmployee(Authentication authentication) {
0037   String username = authentication != null ? authentication.getName() : "admin";
0038   return ResponseEntity.ok(service.getForUsername(username));
0039  }
0040
0041  @GetMapping("/{id}")
0042  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER','MANAGER','EMPLOYEE')")
0043  public EmployeeResponse get(@PathVariable long id) {
0044   return service.getResponse(id);
0045  }
0046
0047  @GetMapping("/{id}/team")
0048  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER','MANAGER','EMPLOYEE')")
0049  public List<EmployeeResponse> getTeam(@PathVariable long id) {
0050   return service.getTeam(id);
0051  }
0052
0053  @PutMapping("/{id}")
0054  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0055  public EmployeeResponse update(@PathVariable long id, @Valid @RequestBody EmployeeRequest r) {
0056   return service.getResponse(service.update(id, r).getId());
0057  }
0058
0059  @DeleteMapping("/{id}")
0060  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0061  public ResponseEntity<?> deactivate(@PathVariable long id) {
0062   service.deactivate(id);
0063   return ResponseEntity.noContent().build();
0064  }
0065 }
```

src/main/java/com/hrgenius/controller/InterviewController.java

```
0001 package com.hrgenius.controller;
0002
0003 import com.hrgenius.entity.Candidate;
0004 import com.hrgenius.entity.Employee;
0005 import com.hrgenius.entity.Interview;
0006 import com.hrgenius.entity.JobOpening;
0007 import com.hrgenius.repository.CandidateRepository;
0008 import com.hrgenius.repository.EmployeeRepository;
0009 import com.hrgenius.repository.InterviewRepository;
0010 import com.hrgenius.repository.JobOpeningRepository;
0011 import org.springframework.http.ResponseEntity;
0012 import org.springframework.security.access.prepost.PreAuthorize;
0013 import org.springframework.web.bind.annotation.*;
0014
0015 import java.time.LocalDateTime;
```

```
0016 import java.util.*;
0017
0018 @RestController
0019 @RequestMapping("/api/v1/interviews")
0020 public class InterviewController {
0021  private final InterviewRepository interviewRepo;
0022  private final CandidateRepository candidateRepo;
0023  private final EmployeeRepository employeeRepo;
0024  private final JobOpeningRepository jobRepo;
0025
0026  public InterviewController(
0027      InterviewRepository interviewRepo,
0028      CandidateRepository candidateRepo,
0029      EmployeeRepository employeeRepo,
0030      JobOpeningRepository jobRepo
0031  ) {
0032   this.interviewRepo = interviewRepo;
0033   this.candidateRepo = candidateRepo;
0034   this.employeeRepo = employeeRepo;
0035   this.jobRepo = jobRepo;
0036  }
0037
0038  @GetMapping
0039  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER','MANAGER')")
0040  public List<Map<String, Object>> list() {
0041   Map<Long, Candidate> candidateMap = new HashMap<>();
0042   for (Candidate c : candidateRepo.findAll()) {
0043    candidateMap.put(c.getId(), c);
0044   }
0045   Map<Long, Employee> empMap = new HashMap<>();
0046   for (Employee e : employeeRepo.findAll()) {
0047    empMap.put(e.getId(), e);
0048   }
0049   Map<Long, JobOpening> jobMap = new HashMap<>();
0050   for (JobOpening j : jobRepo.findAll()) {
0051    jobMap.put(j.getId(), j);
0052   }
0053
0054   List<Map<String, Object>> list = new ArrayList<>();
0055   for (Interview i : interviewRepo.findAll()) {
0056    Map<String, Object> map = new LinkedHashMap<>();
0057    map.put("id", i.getId());
0058    map.put("candidateId", i.getCandidateId());
0059    Candidate cand = candidateMap.get(i.getCandidateId());
0060    map.put("candidateName", cand != null ? cand.getName() : "Candidate #" + i.getCandidateId());
0061
0062    JobOpening job = cand != null && cand.getJobOpeningId() != null ? jobMap.get(cand.getJobOpeningId()) : null;
0063    map.put("jobTitle", job != null ? job.getTitle() : "Recruitment Process");
0064
0065    Employee interviewer = empMap.get(i.getInterviewerId());
0066    map.put("interviewerId", i.getInterviewerId());
0067    map.put("interviewerName", interviewer != null ? interviewer.getName() : "Interviewer #" + i.getInterviewerId());
0068
0069    map.put("scheduledAt", i.getScheduledDate());
0070    map.put("status", i.getResult() != null ? i.getResult() : "SCHEDULED");
0071    map.put("feedback", i.getFeedback());
0072    map.put("mode", i.getMode());
0073    map.put("meetingLink", i.getMeetingLink());
0074    list.add(map);
0075   }
0076   return list;
```

```
 }

 @PostMapping
 @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
 public ResponseEntity<?> schedule(@RequestBody Map<String, Object> body) {
  Interview i = new Interview();
  if (body.get("candidateId") != null) {
   i.setCandidateId(((Number) body.get("candidateId")).longValue());
  }
  if (body.get("interviewerId") != null) {
   i.setInterviewerId(((Number) body.get("interviewerId")).longValue());
  }
  if (body.get("scheduledAt") != null) {
   try {
    i.setScheduledDate(LocalDateTime.parse((String) body.get("scheduledAt")));
   } catch (Exception ex) {
    i.setScheduledDate(LocalDateTime.now().plusDays(1));
   }
  }
  if (body.get("mode") != null) {
   i.setMode((String) body.get("mode"));
  }
  if (body.get("meetingLink") != null) {
   i.setMeetingLink((String) body.get("meetingLink"));
  }
  i.setResult("SCHEDULED");
  return ResponseEntity.status(201).body(interviewRepo.save(i));
 }

 @PutMapping("/{id}/feedback")
 @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER','MANAGER')")
 public ResponseEntity<?> feedback(@PathVariable Long id, @RequestBody Map<String, String> body) {
  Interview i = interviewRepo.findById(id).orElseThrow(() -> new NoSuchElementException("Interview not found"));
  if (body.containsKey("feedback")) {
   i.setFeedback(body.get("feedback"));
  }
  if (body.containsKey("status") && body.get("status") != null) {
   i.setResult(body.get("status"));
   // Also update candidate status if selected or rejected
   candidateRepo.findById(i.getCandidateId()).ifPresent(c -> {
    c.setApplicationStatus(body.get("status"));
    candidateRepo.save(c);
   });
  }
  return ResponseEntity.ok(interviewRepo.save(i));
 }
}
```

src/main/java/com/hrgenius/controller/JobController.java

```
package com.hrgenius.controller;

import com.hrgenius.entity.Candidate;
import com.hrgenius.entity.Department;
import com.hrgenius.entity.JobOpening;
import com.hrgenius.entity.User;
import com.hrgenius.repository.CandidateRepository;
import com.hrgenius.repository.DepartmentRepository;
import com.hrgenius.repository.JobOpeningRepository;
import com.hrgenius.repository.UserRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
```

```
0013 import org.springframework.security.core.Authentication;
0014 import org.springframework.web.bind.annotation.*;
0015
0016 import java.time.LocalDate;
0017 import java.util.*;
0018
0019 @RestController
0020 @RequestMapping("/api/v1/jobs")
0021 public class JobController {
0022  private final JobOpeningRepository jobRepo;
0023  private final DepartmentRepository deptRepo;
0024  private final CandidateRepository candidateRepo;
0025  private final UserRepository userRepo;
0026
0027  public JobController(JobOpeningRepository jobRepo, DepartmentRepository deptRepo, CandidateRepository candidateRepo, UserRepository userRepo) {
0028   this.jobRepo = jobRepo;
0029   this.deptRepo = deptRepo;
0030   this.candidateRepo = candidateRepo;
0031   this.userRepo = userRepo;
0032  }
0033
0034  @GetMapping
0035  public List<Map<String, Object>> list() {
0036   Map<Long, String> deptNames = new HashMap<>();
0037   for (Department d : deptRepo.findAll()) {
0038    deptNames.put(d.getId(), d.getName());
0039   }
0040
0041   List<Map<String, Object>> list = new ArrayList<>();
0042   for (JobOpening j : jobRepo.findAll()) {
0043    Map<String, Object> map = new LinkedHashMap<>();
0044    map.put("id", j.getId());
0045    map.put("title", j.getTitle());
0046    map.put("departmentId", j.getDepartmentId());
0047    map.put("departmentName", j.getDepartmentId() != null ? deptNames.get(j.getDepartmentId()) : "General");
0048    map.put("location", j.getLocation() != null ? j.getLocation() : "Hybrid");
0049    map.put("description", j.getDescription());
0050    map.put("vacancies", j.getVacancies());
0051    map.put("status", j.getStatus());
0052    map.put("postedDate", j.getPostedDate());
0053    list.add(map);
0054   }
0055   return list;
0056  }
0057
0058  @PostMapping
0059  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0060  public ResponseEntity<?> create(@RequestBody Map<String, Object> body) {
0061   JobOpening j = new JobOpening();
0062   j.setTitle((String) body.get("title"));
0063   j.setLocation((String) body.getOrDefault("location", "Hybrid"));
0064   if (body.get("vacancies") != null) {
0065    j.setVacancies(((Number) body.get("vacancies")).intValue());
0066   }
0067   j.setDescription((String) body.get("description"));
0068   if (body.get("departmentId") != null && !String.valueOf(body.get("departmentId")).isBlank()) {
0069    j.setDepartmentId(((Number) body.get("departmentId")).longValue());
0070   }
0071   j.setStatus("OPEN");
0072   j.setPostedDate(LocalDate.now());
0073   return ResponseEntity.status(201).body(jobRepo.save(j));
```

```
 }

 @PutMapping("/{id}")
 @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
 public ResponseEntity<?> update(@PathVariable Long id, @RequestBody Map<String, Object> body) {
  JobOpening j = jobRepo.findById(id).orElseThrow(() -> new NoSuchElementException("Job not found"));
  if (body.containsKey("title")) j.setTitle((String) body.get("title"));
  if (body.containsKey("location")) j.setLocation((String) body.get("location"));
  if (body.containsKey("description")) j.setDescription((String) body.get("description"));
  if (body.containsKey("vacancies") && body.get("vacancies") != null) {
   j.setVacancies(((Number) body.get("vacancies")).intValue());
  }
  if (body.containsKey("departmentId") && body.get("departmentId") != null) {
   j.setDepartmentId(((Number) body.get("departmentId")).longValue());
  }
  if (body.containsKey("status") && body.get("status") != null) {
   j.setStatus((String) body.get("status"));
  }
  return ResponseEntity.ok(jobRepo.save(j));
 }

 @PostMapping("/{id}/apply")
 public ResponseEntity<?> apply(
     @PathVariable Long id,
     @RequestBody(required = false) Map<String, String> body,
     Authentication authentication
 ) {
  JobOpening j = jobRepo.findById(id).orElseThrow(() -> new NoSuchElementException("Job opening not found"));
  String username = authentication != null ? authentication.getName() : "candidate";

  Candidate c = new Candidate();
  c.setJobOpeningId(j.getId());
  c.setApplicationStatus("APPLIED");
  c.setAppliedAt(LocalDate.now());

  String name = (body != null && body.containsKey("name") && !body.get("name").isBlank())
      ? body.get("name")
      : username;
  String email = (body != null && body.containsKey("email") && !body.get("email").isBlank())
      ? body.get("email")
      : (username.contains("@") ? username : username + "@applicant.local");
  String phone = (body != null && body.containsKey("phone")) ? body.get("phone") : "";
  String resume = (body != null && body.containsKey("resume")) ? body.get("resume") : "";

  c.setName(name);
  c.setEmail(email);
  c.setPhone(phone);
  c.setResume(resume);

  candidateRepo.save(c);
  return ResponseEntity.ok(Map.of("message", "Application submitted successfully for " + j.getTitle()));
 }
}
```

src/main/java/com/hrgenius/controller/OnboardingController.java

```
package com.hrgenius.controller;

import com.hrgenius.entity.Employee;
import com.hrgenius.entity.OnboardingTask;
import com.hrgenius.repository.EmployeeRepository;
import com.hrgenius.repository.OnboardingTaskRepository;
```

```
0007 import org.springframework.http.ResponseEntity;
0008 import org.springframework.security.access.prepost.PreAuthorize;
0009 import org.springframework.web.bind.annotation.*;
0010
0011 import java.time.LocalDate;
0012 import java.util.*;
0013
0014 @RestController
0015 @RequestMapping("/api/v1/onboarding")
0016 public class OnboardingController {
0017  private final OnboardingTaskRepository taskRepo;
0018  private final EmployeeRepository employeeRepo;
0019
0020  public OnboardingController(OnboardingTaskRepository taskRepo, EmployeeRepository employeeRepo) {
0021   this.taskRepo = taskRepo;
0022   this.employeeRepo = employeeRepo;
0023  }
0024
0025  @GetMapping
0026  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER','MANAGER')")
0027  public List<Map<String, Object>> getAllTasks(@RequestParam(required = false) Long employeeId) {
0028   Map<Long, Employee> emps = new HashMap<>();
0029   for (Employee e : employeeRepo.findAll()) {
0030    emps.put(e.getId(), e);
0031   }
0032
0033   List<OnboardingTask> tasks = employeeId != null
0034       ? taskRepo.findByEmployeeId(employeeId)
0035       : taskRepo.findAll();
0036
0037   List<Map<String, Object>> list = new ArrayList<>();
0038   for (OnboardingTask t : tasks) {
0039    Employee emp = emps.get(t.getEmployeeId());
0040    Map<String, Object> map = new LinkedHashMap<>();
0041    map.put("id", t.getId());
0042    map.put("employeeId", t.getEmployeeId());
0043    map.put("employeeName", emp != null ? emp.getName() : "Employee #" + t.getEmployeeId());
0044    map.put("employeeCode", emp != null ? emp.getEmployeeCode() : "EMP-" + t.getEmployeeId());
0045    map.put("title", t.getTaskName());
0046    map.put("taskName", t.getTaskName());
0047    map.put("description", t.getDescription());
0048    map.put("status", t.getStatus());
0049    map.put("dueDate", t.getDueDate());
0050    list.add(map);
0051   }
0052   return list;
0053  }
0054
0055  @GetMapping("/employee/{employeeId}")
0056  public List<Map<String, Object>> getEmployeeTasks(@PathVariable Long employeeId) {
0057   Employee emp = employeeRepo.findById(employeeId).orElse(null);
0058   String empName = emp != null ? emp.getName() : "Employee #" + employeeId;
0059
0060   List<Map<String, Object>> list = new ArrayList<>();
0061   for (OnboardingTask t : taskRepo.findByEmployeeId(employeeId)) {
0062    Map<String, Object> map = new LinkedHashMap<>();
0063    map.put("id", t.getId());
0064    map.put("employeeId", t.getEmployeeId());
0065    map.put("employeeName", empName);
0066    map.put("title", t.getTaskName());
0067    map.put("taskName", t.getTaskName());
```

```
0068    map.put("description", t.getDescription());
0069    map.put("status", t.getStatus());
0070    map.put("dueDate", t.getDueDate());
0071    list.add(map);
0072   }
0073   return list;
0074  }
0075
0076  @PostMapping("/tasks")
0077  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0078  public ResponseEntity<?> createTask(@RequestBody Map<String, Object> body) {
0079   OnboardingTask t = new OnboardingTask();
0080   if (body.get("employeeId") != null) {
0081    t.setEmployeeId(((Number) body.get("employeeId")).longValue());
0082   }
0083   t.setTaskName((String) body.getOrDefault("title", "Onboarding Task"));
0084   t.setDescription((String) body.get("description"));
0085   if (body.get("dueDate") != null && !String.valueOf(body.get("dueDate")).isBlank()) {
0086    try {
0087     t.setDueDate(LocalDate.parse((String) body.get("dueDate")));
0088    } catch (Exception ignored) {
0089     t.setDueDate(LocalDate.now().plusDays(7));
0090    }
0091   } else {
0092    t.setDueDate(LocalDate.now().plusDays(7));
0093   }
0094   t.setStatus("PENDING");
0095   return ResponseEntity.status(201).body(taskRepo.save(t));
0096  }
0097
0098  @PutMapping("/tasks/{id}")
0099  public ResponseEntity<?> updateTask(@PathVariable Long id, @RequestBody Map<String, String> body) {
0100   OnboardingTask t = taskRepo.findById(id).orElseThrow(() -> new NoSuchElementException("Task not found"));
0101   String status = body.get("status");
0102   if (status != null) {
0103    t.setStatus(status);
0104    taskRepo.save(t);
0105   }
0106   return ResponseEntity.ok(t);
0107  }
0108 }
```

src/main/java/com/hrgenius/controller/PayrollController.java

```
0001 package com.hrgenius.controller;
0002
0003 import com.hrgenius.entity.Employee;
0004 import com.hrgenius.entity.Payroll;
0005 import com.hrgenius.entity.SalaryStructure;
0006 import com.hrgenius.repository.EmployeeRepository;
0007 import com.hrgenius.repository.PayrollRepository;
0008 import com.hrgenius.repository.SalaryStructureRepository;
0009 import com.hrgenius.service.PayrollCalculator;
0010 import com.lowagie.text.Document;
0011 import com.lowagie.text.Element;
0012 import com.lowagie.text.Font;
0013 import com.lowagie.text.FontFactory;
0014 import com.lowagie.text.PageSize;
0015 import com.lowagie.text.Paragraph;
0016 import com.lowagie.text.Phrase;
0017 import com.lowagie.text.pdf.PdfPCell;
0018 import com.lowagie.text.pdf.PdfPTable;
```

```
0019 import com.lowagie.text.pdf.PdfWriter;
0020 import org.springframework.http.HttpHeaders;
0021 import org.springframework.http.MediaType;
0022 import org.springframework.http.ResponseEntity;
0023 import org.springframework.security.access.prepost.PreAuthorize;
0024 import org.springframework.web.bind.annotation.*;
0025
0026 import java.awt.Color;
0027 import java.io.ByteArrayOutputStream;
0028 import java.math.BigDecimal;
0029 import java.time.Month;
0030 import java.util.*;
0031
0032 @RestController
0033 @RequestMapping("/api/v1/payroll")
0034 public class PayrollController {
0035  private final PayrollRepository payrollRepo;
0036  private final EmployeeRepository employeeRepo;
0037  private final SalaryStructureRepository salaryRepo;
0038
0039  public PayrollController(
0040      PayrollRepository payrollRepo,
0041      EmployeeRepository employeeRepo,
0042      SalaryStructureRepository salaryRepo
0043  ) {
0044   this.payrollRepo = payrollRepo;
0045   this.employeeRepo = employeeRepo;
0046   this.salaryRepo = salaryRepo;
0047  }
0048
0049  @GetMapping
0050  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0051  public java.util.List<Map<String, Object>> getAllPayroll() {
0052   Map<Long, Employee> emps = new HashMap<>();
0053   for (Employee e : employeeRepo.findAll()) {
0054    emps.put(e.getId(), e);
0055   }
0056
0057   java.util.List<Map<String, Object>> result = new ArrayList<>();
0058   for (Payroll p : payrollRepo.findAll()) {
0059    Employee emp = emps.get(p.getEmployeeId());
0060    Map<String, Object> map = new LinkedHashMap<>();
0061    map.put("id", p.getId());
0062    map.put("employeeId", p.getEmployeeId());
0063    map.put("employeeCode", emp != null ? emp.getEmployeeCode() : "EMP-" + p.getEmployeeId());
0064    map.put("employeeName", emp != null ? emp.getName() : "Employee #" + p.getEmployeeId());
0065    map.put("designation", emp != null ? emp.getDesignation() : "Staff");
0066    map.put("month", p.getMonth());
0067    map.put("year", p.getYear());
0068    map.put("grossPay", p.getGrossPay());
0069    map.put("grossSalary", p.getGrossPay());
0070    map.put("pf", p.getPf());
0071    map.put("tds", p.getTds());
0072    map.put("otherDeductions", p.getOtherDeductions());
0073    map.put("totalDeductions", p.getTotalDeductions());
0074    map.put("netPay", p.getNetPay());
0075    map.put("status", p.getStatus());
0076    result.add(map);
0077   }
0078   return result;
0079  }
```

```
0080
0081  @GetMapping("/{employeeId}")
0082  public java.util.List<Payroll> getPayrollHistory(@PathVariable Long employeeId) {
0083   return payrollRepo.findByEmployeeIdOrderByYearDescMonthDesc(employeeId);
0084  }
0085
0086  @PostMapping("/run")
0087  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0088  public ResponseEntity<?> runPayroll(@RequestBody Map<String, Object> body) {
0089   int month = ((Number) body.get("month")).intValue();
0090   int year = ((Number) body.get("year")).intValue();
0091
0092   java.util.List<Employee> activeEmployees = employeeRepo.findAll().stream()
0093       .filter(e -> "ACTIVE".equalsIgnoreCase(e.getStatus()))
0094       .toList();
0095
0096   int processedCount = 0;
0097   int skippedCount = 0;
0098   List<String> skippedNames = new ArrayList<>();
0099
0100   for (Employee emp : activeEmployees) {
0101    Optional<SalaryStructure> opt = salaryRepo.findByEmployeeId(emp.getId());
0102    SalaryStructure structure;
0103    if (opt.isPresent()) {
0104     structure = opt.get();
0105    } else if (body.containsKey("defaultBasic") && body.get("defaultBasic") != null) {
0106     // User explicitly provided defaults in the payroll run modal
0107     BigDecimal defBasic = new BigDecimal(String.valueOf(body.get("defaultBasic")));
0108     BigDecimal defHra = body.containsKey("defaultHra") ? new BigDecimal(String.valueOf(body.get("defaultHra"))) : defBasic.multiply(new BigDecimal("0.40"));
0109     BigDecimal defOther = body.containsKey("defaultAllowances") ? new BigDecimal(String.valueOf(body.get("defaultAllowances"))) : BigDecimal.ZERO;
0110     BigDecimal defDeductions = body.containsKey("defaultDeductions") ? new BigDecimal(String.valueOf(body.get("defaultDeductions"))) : defBasic.multiply(new BigDecimal("0.12"));
0111
0112     SalaryStructure ns = new SalaryStructure();
0113     ns.setEmployeeId(emp.getId());
0114     ns.setBasic(defBasic);
0115     ns.setHra(defHra);
0116     ns.setOtherAllowances(defOther);
0117     ns.setDeductions(defDeductions);
0118     structure = salaryRepo.save(ns);
0119    } else {
0120     skippedCount++;
0121     skippedNames.add(emp.getName());
0122     continue;
0123    }
0124
0125    BigDecimal basic = structure.getBasic() != null ? structure.getBasic() : BigDecimal.ZERO;
0126    BigDecimal hra = structure.getHra() != null ? structure.getHra() : BigDecimal.ZERO;
0127    BigDecimal allowances = structure.getOtherAllowances() != null ? structure.getOtherAllowances() : BigDecimal.ZERO;
0128    BigDecimal monthlyGross = basic.add(hra).add(allowances);
0129    BigDecimal tds = PayrollCalculator.monthlyTds(monthlyGross);
0130    BigDecimal otherDeductions = structure.getDeductions() != null ? structure.getDeductions() : BigDecimal.ZERO;
0131
0132    PayrollCalculator.Result res = PayrollCalculator.calculate(basic, hra, allowances, otherDeductions, tds);
0133
0134    Payroll p = payrollRepo.findByEmployeeIdAndMonthAndYear(emp.getId(), month, year)
0135        .orElseGet(() -> {
0136         Payroll np = new Payroll();
0137         np.setEmployeeId(emp.getId());
0138         np.setMonth(month);
0139         np.setYear(year);
0140         return np;
```

```
       });

    p.setGrossPay(res.gross());
    p.setPf(res.pf());
    p.setTds(res.tds());
    p.setOtherDeductions(res.otherDeduction());
    p.setTotalDeductions(res.totalDeductions());
    p.setNetPay(res.net());
    p.setStatus("PROCESSED");

    payrollRepo.save(p);
    processedCount++;
   }

   String note = skippedCount > 0
       ? " (" + skippedCount + " employees skipped due to missing salary structures: " + String.join(", ", skippedNames) + ")"
       : "";

   return ResponseEntity.ok(Map.of(
       "message", "Payroll executed successfully. " + processedCount + " employee(s) processed" + note,
       "processedEmployees", processedCount,
       "skippedEmployees", skippedCount,
       "month", month,
       "year", year
   ));
  }

  @GetMapping("/{employeeId}/payslip")
  public ResponseEntity<byte[]> getPayslip(
      @PathVariable Long employeeId,
      @RequestParam int month,
      @RequestParam int year
  ) {
   Employee emp = employeeRepo.findById(employeeId)
       .orElseThrow(() -> new NoSuchElementException("Employee not found with ID " + employeeId));

   Payroll p = payrollRepo.findByEmployeeIdAndMonthAndYear(employeeId, month, year)
       .orElseGet(() -> {
        SalaryStructure ss = salaryRepo.findByEmployeeId(employeeId)
            .orElseThrow(() -> new NoSuchElementException("No salary structure configured for employee " + emp.getName() + " (ID " + employeeId + "). Please configure salary structure
 first."));

        BigDecimal basic = ss.getBasic() != null ? ss.getBasic() : BigDecimal.ZERO;
        BigDecimal hra = ss.getHra() != null ? ss.getHra() : BigDecimal.ZERO;
        BigDecimal allowances = ss.getOtherAllowances() != null ? ss.getOtherAllowances() : BigDecimal.ZERO;
        BigDecimal monthlyGross = basic.add(hra).add(allowances);
        BigDecimal tds = PayrollCalculator.monthlyTds(monthlyGross);
        BigDecimal deductions = ss.getDeductions() != null ? ss.getDeductions() : BigDecimal.ZERO;

        PayrollCalculator.Result r = PayrollCalculator.calculate(basic, hra, allowances, deductions, tds);
        Payroll np = new Payroll();
        np.setEmployeeId(employeeId);
        np.setMonth(month);
        np.setYear(year);
        np.setGrossPay(r.gross());
        np.setPf(r.pf());
        np.setTds(r.tds());
        np.setOtherDeductions(r.otherDeduction());
        np.setTotalDeductions(r.totalDeductions());
        np.setNetPay(r.net());
        np.setStatus("PROCESSED");
```

```
0201        return payrollRepo.save(np);
0202       });
0203
0204   try {
0205    ByteArrayOutputStream out = new ByteArrayOutputStream();
0206    Document doc = new Document(PageSize.A4, 40, 40, 40, 40);
0207    PdfWriter.getInstance(doc, out);
0208    doc.open();
0209
0210    Font titleFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 18, new Color(16, 43, 78));
0211    Font headerFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 12, new Color(40, 117, 199));
0212    Font boldFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 10, Color.BLACK);
0213    Font regularFont = FontFactory.getFont(FontFactory.HELVETICA, 10, Color.DARK_GRAY);
0214
0215    Paragraph title = new Paragraph("HRGenius Enterprise - Payslip", titleFont);
0216    title.setAlignment(Element.ALIGN_CENTER);
0217    doc.add(title);
0218
0219    String monthName = Month.of(Math.max(1, Math.min(12, month))).name();
0220    Paragraph period = new Paragraph("Pay Period: " + monthName + " " + year, regularFont);
0221    period.setAlignment(Element.ALIGN_CENTER);
0222    period.setSpacingAfter(20);
0223    doc.add(period);
0224
0225    // Employee Info Table
0226    PdfPTable infoTable = new PdfPTable(2);
0227    infoTable.setWidthPercentage(100);
0228    infoTable.setSpacingAfter(15);
0229
0230    addCell(infoTable, "Employee Name:", boldFont);
0231    addCell(infoTable, emp.getName(), regularFont);
0232    addCell(infoTable, "Employee Code:", boldFont);
0233    addCell(infoTable, emp.getEmployeeCode(), regularFont);
0234    addCell(infoTable, "Designation:", boldFont);
0235    addCell(infoTable, emp.getDesignation(), regularFont);
0236    addCell(infoTable, "Email:", boldFont);
0237    addCell(infoTable, emp.getEmail(), regularFont);
0238
0239    doc.add(infoTable);
0240
0241    // Salary Breakdown Table
0242    PdfPTable payTable = new PdfPTable(4);
0243    payTable.setWidthPercentage(100);
0244    payTable.setWidths(new float[]{30, 20, 30, 20});
0245    payTable.setSpacingAfter(20);
0246
0247    addHeaderCell(payTable, "Earnings", headerFont);
0248    addHeaderCell(payTable, "Amount (INR)", headerFont);
0249    addHeaderCell(payTable, "Deductions", headerFont);
0250    addHeaderCell(payTable, "Amount (INR)", headerFont);
0251
0252    addCell(payTable, "Gross Earnings", regularFont);
0253    addCell(payTable, p.getGrossPay().toString(), regularFont);
0254    addCell(payTable, "Provident Fund (PF)", regularFont);
0255    addCell(payTable, p.getPf().toString(), regularFont);
0256
0257    addCell(payTable, "", regularFont);
0258    addCell(payTable, "", regularFont);
0259    addCell(payTable, "Income Tax (TDS)", regularFont);
0260    addCell(payTable, p.getTds().toString(), regularFont);
0261
```

```
0262    addCell(payTable, "", regularFont);
0263    addCell(payTable, "", regularFont);
0264    addCell(payTable, "Other Deductions", regularFont);
0265    addCell(payTable, p.getOtherDeductions().toString(), regularFont);
0266
0267    addCell(payTable, "Total Gross Pay", boldFont);
0268    addCell(payTable, p.getGrossPay().toString(), boldFont);
0269    addCell(payTable, "Total Deductions", boldFont);
0270    addCell(payTable, p.getTotalDeductions().toString(), boldFont);
0271
0272    doc.add(payTable);
0273
0274    // Net Pay Highlight
0275    Paragraph net = new Paragraph("Net Pay: INR " + p.getNetPay().toString(), FontFactory.getFont(FontFactory.HELVETICA_BOLD, 14, new Color(22, 132, 91)));
0276    net.setAlignment(Element.ALIGN_RIGHT);
0277    net.setSpacingAfter(30);
0278    doc.add(net);
0279
0280    Paragraph footer = new Paragraph("This is a computer-generated payslip and does not require a physical signature.", FontFactory.getFont(FontFactory.HELVETICA_OBLIQUE, 8, Color.GRA
Y));
0281    footer.setAlignment(Element.ALIGN_CENTER);
0282    doc.add(footer);
0283
0284    doc.close();
0285
0286    byte[] pdfBytes = out.toByteArray();
0287    return ResponseEntity.ok()
0288        .contentType(MediaType.APPLICATION_PDF)
0289        .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"HRGenius-payslip-" + month + "-" + year + ".pdf\"")
0290        .body(pdfBytes);
0291   } catch (NoSuchElementException nse) {
0292    throw nse;
0293   } catch (Exception ex) {
0294    throw new RuntimeException("Error generating PDF payslip: " + ex.getMessage(), ex);
0295   }
0296  }
0297
0298  private void addCell(PdfPTable table, String text, Font font) {
0299   PdfPCell cell = new PdfPCell(new Phrase(text, font));
0300   cell.setPadding(6);
0301   cell.setBorderColor(new Color(230, 235, 240));
0302   table.addCell(cell);
0303  }
0304
0305  private void addHeaderCell(PdfPTable table, String text, Font font) {
0306   PdfPCell cell = new PdfPCell(new Phrase(text, font));
0307   cell.setPadding(8);
0308   cell.setBackgroundColor(new Color(245, 248, 252));
0309   cell.setBorderColor(new Color(220, 228, 236));
0310   table.addCell(cell);
0311  }
0312 }
```

src/main/java/com/hrgenius/controller/PerformanceController.java

```
0001 package com.hrgenius.controller;
0002
0003 import com.hrgenius.entity.*;
0004 import com.hrgenius.repository.*;
0005 import org.springframework.http.ResponseEntity;
0006 import org.springframework.security.access.prepost.PreAuthorize;
0007 import org.springframework.security.core.Authentication;
```

```
0008 import org.springframework.web.bind.annotation.*;
0009
0010 import java.time.LocalDate;
0011 import java.util.*;
0012
0013 @RestController
0014 @RequestMapping("/api/v1/performance")
0015 public class PerformanceController {
0016  private final PerformanceCycleRepository cycleRepo;
0017  private final GoalRepository goalRepo;
0018  private final PerformanceReviewRepository reviewRepo;
0019  private final EmployeeRepository employeeRepo;
0020
0021  public PerformanceController(
0022      PerformanceCycleRepository cycleRepo,
0023      GoalRepository goalRepo,
0024      PerformanceReviewRepository reviewRepo,
0025      EmployeeRepository employeeRepo
0026  ) {
0027   this.cycleRepo = cycleRepo;
0028   this.goalRepo = goalRepo;
0029   this.reviewRepo = reviewRepo;
0030   this.employeeRepo = employeeRepo;
0031  }
0032
0033  @GetMapping("/cycles")
0034  public List<PerformanceCycle> listCycles() {
0035   return cycleRepo.findAll();
0036  }
0037
0038  @PostMapping("/cycles")
0039  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0040  public ResponseEntity<?> createCycle(@RequestBody Map<String, Object> body) {
0041   PerformanceCycle c = new PerformanceCycle();
0042   c.setName((String) body.get("name"));
0043   if (body.get("startDate") != null) {
0044    c.setStartDate(LocalDate.parse((String) body.get("startDate")));
0045   } else {
0046    c.setStartDate(LocalDate.now());
0047   }
0048   if (body.get("endDate") != null) {
0049    c.setEndDate(LocalDate.parse((String) body.get("endDate")));
0050   } else {
0051    c.setEndDate(LocalDate.now().plusMonths(6));
0052   }
0053   c.setStatus("ACTIVE");
0054   return ResponseEntity.status(201).body(cycleRepo.save(c));
0055  }
0056
0057  @GetMapping("/goals")
0058  public List<Map<String, Object>> listGoals(@RequestParam(required = false) Long employeeId) {
0059   Map<Long, Employee> emps = new HashMap<>();
0060   for (Employee e : employeeRepo.findAll()) {
0061    emps.put(e.getId(), e);
0062   }
0063
0064   List<Goal> goals = employeeId != null
0065       ? goalRepo.findByEmployeeId(employeeId)
0066       : goalRepo.findAll();
0067
0068   List<Map<String, Object>> result = new ArrayList<>();
```

```
  for (Goal g : goals) {
   Employee emp = emps.get(g.getEmployeeId());
   Map<String, Object> map = new LinkedHashMap<>();
   map.put("id", g.getId());
   map.put("employeeId", g.getEmployeeId());
   map.put("employeeName", emp != null ? emp.getName() : "Employee #" + g.getEmployeeId());
   map.put("title", g.getTitle());
   map.put("description", g.getDescription());
   map.put("status", g.getStatus());
   map.put("dueDate", g.getDueDate());
   result.add(map);
  }
  return result;
 }

 @PostMapping("/goals")
 @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER','MANAGER')")
 public ResponseEntity<?> createGoal(@RequestBody Map<String, Object> body, Authentication auth) {
  Goal g = new Goal();
  if (body.get("employeeId") != null) {
   g.setEmployeeId(((Number) body.get("employeeId")).longValue());
  }
  g.setTitle((String) body.get("title"));
  g.setDescription((String) body.get("description"));
  if (body.get("dueDate") != null) {
   try {
    g.setDueDate(LocalDate.parse((String) body.get("dueDate")));
   } catch (Exception ignored) {
    g.setDueDate(LocalDate.now().plusMonths(3));
   }
  }
  g.setStatus("IN_PROGRESS");
  // Link to active cycle if any
  cycleRepo.findAll().stream().findFirst().ifPresent(c -> g.setCycleId(c.getId()));
  return ResponseEntity.status(201).body(goalRepo.save(g));
 }

 @PostMapping("/self-assessments")
 public ResponseEntity<?> submitSelfAssessment(@RequestBody Map<String, Object> body, Authentication auth) {
  Long cycleId = ((Number) body.get("cycleId")).longValue();
  String assessmentText = (String) body.get("selfAssessment");

  String username = auth != null ? auth.getName() : "employee";
  Employee emp = employeeRepo.findByEmail(username)
      .or(() -> employeeRepo.findAll().stream().findFirst())
      .orElseThrow(() -> new NoSuchElementException("Employee profile not found"));

  PerformanceReview review = reviewRepo.findByEmployeeId(emp.getId()).stream()
      .filter(r -> r.getCycleId().equals(cycleId))
      .findFirst()
      .orElseGet(() -> {
       PerformanceReview nr = new PerformanceReview();
       nr.setEmployeeId(emp.getId());
       nr.setCycleId(cycleId);
       return nr;
      });

  review.setSelfAssessment(assessmentText);
  review.setStatus("ASSESSED");
  reviewRepo.save(review);
  return ResponseEntity.ok(Map.of("message", "Self-assessment submitted successfully"));
```

```
0130  }
0131
0132  @PostMapping("/reviews")
0133  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER','MANAGER')")
0134  public ResponseEntity<?> createReview(@RequestBody Map<String, Object> body, Authentication auth) {
0135   Long employeeId = ((Number) body.get("employeeId")).longValue();
0136   Long cycleId = ((Number) body.get("cycleId")).longValue();
0137   Integer rating = ((Number) body.get("rating")).intValue();
0138   String comments = (String) body.get("comments");
0139
0140   PerformanceReview review = reviewRepo.findByEmployeeId(employeeId).stream()
0141       .filter(r -> r.getCycleId().equals(cycleId))
0142       .findFirst()
0143       .orElseGet(() -> {
0144        PerformanceReview nr = new PerformanceReview();
0145        nr.setEmployeeId(employeeId);
0146        nr.setCycleId(cycleId);
0147        return nr;
0148       });
0149
0150   review.setRating(rating);
0151   review.setComments(comments);
0152   review.setStatus("COMPLETED");
0153   return ResponseEntity.ok(reviewRepo.save(review));
0154  }
0155
0156  @GetMapping("/reviews")
0157  public List<Map<String, Object>> getAllReviews() {
0158   Map<Long, String> cycleNames = new HashMap<>();
0159   for (PerformanceCycle c : cycleRepo.findAll()) {
0160    cycleNames.put(c.getId(), c.getName());
0161   }
0162
0163   Map<Long, Employee> emps = new HashMap<>();
0164   for (Employee e : employeeRepo.findAll()) {
0165    emps.put(e.getId(), e);
0166   }
0167
0168   List<Map<String, Object>> list = new ArrayList<>();
0169   for (PerformanceReview r : reviewRepo.findAll()) {
0170    Employee emp = emps.get(r.getEmployeeId());
0171    Map<String, Object> map = new LinkedHashMap<>();
0172    map.put("id", r.getId());
0173    map.put("cycleId", r.getCycleId());
0174    map.put("cycleName", cycleNames.getOrDefault(r.getCycleId(), "Cycle #" + r.getCycleId()));
0175    map.put("employeeId", r.getEmployeeId());
0176    map.put("employeeName", emp != null ? emp.getName() : "Employee #" + r.getEmployeeId());
0177    map.put("rating", r.getRating() != null ? r.getRating() : "—");
0178    map.put("status", r.getStatus() != null ? r.getStatus() : "SUBMITTED");
0179    map.put("comments", r.getComments() != null ? r.getComments() : r.getSelfAssessment());
0180    map.put("selfAssessment", r.getSelfAssessment());
0181    map.put("feedback", r.getComments());
0182    list.add(map);
0183   }
0184   return list;
0185  }
0186
0187  @GetMapping("/reviews/me")
0188  public List<Map<String, Object>> getMyReviews(Authentication auth) {
0189   String username = auth != null ? auth.getName() : "employee";
0190   Employee emp = employeeRepo.findByEmail(username)
```

```
      .or(() -> employeeRepo.findAll().stream().filter(e -> username.equalsIgnoreCase(e.getEmail())).findFirst())
      .orElseGet(() -> employeeRepo.findAll().stream().findFirst().orElse(null));

  if (emp == null) return List.of();
  return getReviews(emp.getId());
 }

 @GetMapping("/reviews/{employeeId}")
 public List<Map<String, Object>> getReviews(@PathVariable Long employeeId) {
  Map<Long, String> cycleNames = new HashMap<>();
  for (PerformanceCycle c : cycleRepo.findAll()) {
   cycleNames.put(c.getId(), c.getName());
  }

  Employee emp = employeeRepo.findById(employeeId).orElse(null);
  String empName = emp != null ? emp.getName() : "Employee #" + employeeId;

  List<Map<String, Object>> list = new ArrayList<>();
  for (PerformanceReview r : reviewRepo.findByEmployeeId(employeeId)) {
   Map<String, Object> map = new LinkedHashMap<>();
   map.put("id", r.getId());
   map.put("cycleId", r.getCycleId());
   map.put("cycleName", cycleNames.getOrDefault(r.getCycleId(), "Cycle #" + r.getCycleId()));
   map.put("employeeId", r.getEmployeeId());
   map.put("employeeName", empName);
   map.put("rating", r.getRating() != null ? r.getRating() : "—");
   map.put("status", r.getStatus() != null ? r.getStatus() : "SUBMITTED");
   map.put("comments", r.getComments() != null ? r.getComments() : r.getSelfAssessment());
   map.put("selfAssessment", r.getSelfAssessment());
   map.put("feedback", r.getComments());
   list.add(map);
  }
  return list;
 }
}
```

src/main/java/com/hrgenius/controller/SalaryStructureController.java

```
package com.hrgenius.controller;

import com.hrgenius.entity.Department;
import com.hrgenius.entity.Employee;
import com.hrgenius.entity.SalaryStructure;
import com.hrgenius.repository.DepartmentRepository;
import com.hrgenius.repository.EmployeeRepository;
import com.hrgenius.repository.SalaryStructureRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.math.BigDecimal;
import java.util.*;

@RestController
@RequestMapping("/api/v1/salary-structures")
public class SalaryStructureController {
 private final SalaryStructureRepository salaryRepo;
 private final EmployeeRepository employeeRepo;
 private final DepartmentRepository deptRepo;

 public SalaryStructureController(SalaryStructureRepository salaryRepo, EmployeeRepository employeeRepo, DepartmentRepository deptRepo) {
```

```
  this.salaryRepo = salaryRepo;
  this.employeeRepo = employeeRepo;
  this.deptRepo = deptRepo;
 }

 @GetMapping
 @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER','MANAGER')")
 public List<Map<String, Object>> listAll() {
  Map<Long, Department> depts = new HashMap<>();
  for (Department d : deptRepo.findAll()) {
   depts.put(d.getId(), d);
  }

  Map<Long, Employee> emps = new HashMap<>();
  for (Employee e : employeeRepo.findAll()) {
   emps.put(e.getId(), e);
  }

  List<Map<String, Object>> result = new ArrayList<>();
  for (SalaryStructure s : salaryRepo.findAll()) {
   Employee emp = emps.get(s.getEmployeeId());
   Map<String, Object> map = new LinkedHashMap<>();
   map.put("id", s.getId());
   map.put("employeeId", s.getEmployeeId());
   map.put("employeeCode", emp != null ? emp.getEmployeeCode() : "EMP-" + s.getEmployeeId());
   map.put("employeeName", emp != null ? emp.getName() : "Employee #" + s.getEmployeeId());
   map.put("designation", emp != null ? emp.getDesignation() : "Staff");
   String deptName = (emp != null && emp.getDepartmentId() != null && depts.containsKey(emp.getDepartmentId()))
       ? depts.get(emp.getDepartmentId()).getName() : "General";
   map.put("departmentName", deptName);
   BigDecimal basic = s.getBasic() != null ? s.getBasic() : BigDecimal.ZERO;
   BigDecimal hra = s.getHra() != null ? s.getHra() : BigDecimal.ZERO;
   BigDecimal other = s.getOtherAllowances() != null ? s.getOtherAllowances() : BigDecimal.ZERO;
   BigDecimal deductions = s.getDeductions() != null ? s.getDeductions() : BigDecimal.ZERO;
   BigDecimal gross = basic.add(hra).add(other);
   BigDecimal net = gross.subtract(deductions);

   map.put("basic", basic);
   map.put("basicPay", basic);
   map.put("hra", hra);
   map.put("otherAllowances", other);
   map.put("allowances", other);
   map.put("deductions", deductions);
   map.put("grossPay", gross);
   map.put("netPay", net);
   result.add(map);
  }
  return result;
 }

 @GetMapping("/me")
 public ResponseEntity<?> getMyStructure(Authentication auth) {
  String username = auth != null ? auth.getName() : "admin";
  Employee emp = employeeRepo.findByEmail(username)
      .or(() -> employeeRepo.findAll().stream().filter(e -> username.equalsIgnoreCase(e.getEmail())).findFirst())
      .orElseGet(() -> employeeRepo.findAll().stream().findFirst().orElse(null));

  if (emp == null) {
   return ResponseEntity.badRequest().body(Map.of("message", "No employee profile found"));
  }
  return getStructure(emp.getId());
```

```
0086  }
0087
0088  @GetMapping("/{employeeId}")
0089  public ResponseEntity<?> getStructure(@PathVariable Long employeeId) {
0090   SalaryStructure s = salaryRepo.findByEmployeeId(employeeId)
0091       .orElseThrow(() -> new NoSuchElementException("Salary structure not defined for employee ID " + employeeId));
0092
0093   Employee emp = employeeRepo.findById(employeeId).orElse(null);
0094
0095   BigDecimal basic = s.getBasic() != null ? s.getBasic() : BigDecimal.ZERO;
0096   BigDecimal hra = s.getHra() != null ? s.getHra() : BigDecimal.ZERO;
0097   BigDecimal other = s.getOtherAllowances() != null ? s.getOtherAllowances() : BigDecimal.ZERO;
0098   BigDecimal deductions = s.getDeductions() != null ? s.getDeductions() : BigDecimal.ZERO;
0099   BigDecimal gross = basic.add(hra).add(other);
0100   BigDecimal net = gross.subtract(deductions);
0101
0102   Map<String, Object> map = new LinkedHashMap<>();
0103   map.put("id", s.getId());
0104   map.put("employeeId", s.getEmployeeId());
0105   map.put("employeeCode", emp != null ? emp.getEmployeeCode() : "EMP-" + employeeId);
0106   map.put("employeeName", emp != null ? emp.getName() : "Employee #" + employeeId);
0107   map.put("designation", emp != null ? emp.getDesignation() : "Staff");
0108   map.put("basic", basic);
0109   map.put("basicPay", basic);
0110   map.put("hra", hra);
0111   map.put("otherAllowances", other);
0112   map.put("allowances", other);
0113   map.put("deductions", deductions);
0114   map.put("grossPay", gross);
0115   map.put("netPay", net);
0116   return ResponseEntity.ok(map);
0117  }
0118
0119  @PostMapping
0120  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0121  public ResponseEntity<?> createOrUpdate(@RequestBody Map<String, Object> body) {
0122   if (body.get("employeeId") == null) {
0123    return ResponseEntity.badRequest().body(Map.of("message", "Employee ID is required"));
0124   }
0125   Long employeeId = ((Number) body.get("employeeId")).longValue();
0126   Employee emp = employeeRepo.findById(employeeId)
0127       .orElseThrow(() -> new NoSuchElementException("Employee not found with ID " + employeeId));
0128
0129   SalaryStructure s = salaryRepo.findByEmployeeId(employeeId).orElseGet(() -> {
0130    SalaryStructure ns = new SalaryStructure();
0131    ns.setEmployeeId(employeeId);
0132    return ns;
0133   });
0134
0135   BigDecimal basic = body.get("basic") != null ? new BigDecimal(String.valueOf(body.get("basic"))) : BigDecimal.ZERO;
0136   BigDecimal hra = body.get("hra") != null ? new BigDecimal(String.valueOf(body.get("hra"))) : BigDecimal.ZERO;
0137   BigDecimal other = body.get("otherAllowances") != null ? new BigDecimal(String.valueOf(body.get("otherAllowances"))) : BigDecimal.ZERO;
0138
0139   // Use user-defined deductions if specified by user; otherwise standard PF (12% of basic)
0140   BigDecimal deductions;
0141   if (body.get("deductions") != null && !String.valueOf(body.get("deductions")).isBlank()) {
0142    deductions = new BigDecimal(String.valueOf(body.get("deductions")));
0143   } else {
0144    deductions = basic.multiply(new BigDecimal("0.12")).setScale(2, java.math.RoundingMode.HALF_UP);
0145   }
0146
```

```
0147   s.setBasic(basic);
0148   s.setHra(hra);
0149   s.setOtherAllowances(other);
0150   s.setDeductions(deductions);
0151
0152   SalaryStructure saved = salaryRepo.save(s);
0153   return ResponseEntity.ok(Map.of(
0154       "message", "Salary structure saved successfully for " + emp.getName(),
0155       "id", saved.getId(),
0156       "employeeId", saved.getEmployeeId()
0157   ));
0158  }
0159
0160  @DeleteMapping("/{id}")
0161  @PreAuthorize("hasAnyRole('ADMIN','HR_MANAGER')")
0162  public ResponseEntity<?> delete(@PathVariable Long id) {
0163   salaryRepo.deleteById(id);
0164   return ResponseEntity.noContent().build();
0165  }
0166 }
```

src/main/java/com/hrgenius/dto/EmployeeRequest.java

```
0001 package com.hrgenius.dto;
0002 import jakarta.validation.constraints.*; import java.time.LocalDate;
0003 public record EmployeeRequest(@NotBlank @Size(max=150) String name,@NotBlank @Email String email,
0004  @Size(max=20) String phone,@NotBlank @Size(max=100) String designation,@NotNull Long departmentId,
0005  @NotNull LocalDate dateOfJoining, Long managerId) {}
```

src/main/java/com/hrgenius/dto/EmployeeResponse.java

```
0001 package com.hrgenius.dto;
0002
0003 import com.hrgenius.entity.Employee;
0004 import java.time.LocalDate;
0005
0006 public record EmployeeResponse(
0007     Long id,
0008     String employeeCode,
0009     String name,
0010     String email,
0011     String phone,
0012     LocalDate dateOfJoining,
0013     String designation,
0014     String status,
0015     Long departmentId,
0016     String departmentName,
0017     Long managerId
0018 ) {
0019  public static EmployeeResponse from(Employee e) {
0020   return new EmployeeResponse(e.getId(), e.getEmployeeCode(), e.getName(), e.getEmail(), e.getPhone(), e.getDateOfJoining(), e.getDesignation(), e.getStatus(), e.getDepartmentId(), n
ull, e.getManagerId());
0021  }
0022  public static EmployeeResponse from(Employee e, String departmentName) {
0023   return new EmployeeResponse(e.getId(), e.getEmployeeCode(), e.getName(), e.getEmail(), e.getPhone(), e.getDateOfJoining(), e.getDesignation(), e.getStatus(), e.getDepartmentId(), d
epartmentName, e.getManagerId());
0024  }
0025 }
```

src/main/java/com/hrgenius/dto/LoginRequest.java

```
0001 package com.hrgenius.dto;
0002 import jakarta.validation.constraints.NotBlank;
```

```
0003 public record LoginRequest(@NotBlank String username,@NotBlank String password) {}
```

src/main/java/com/hrgenius/entity/Candidate.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.time.*;
0005
0006 @Entity
0007 @Table(name="candidate")
0008 @Getter @Setter @NoArgsConstructor
0009 public class Candidate {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(nullable=false,length=150) private String name;
0012  @Column(nullable=false,length=150) private String email;
0013  @Column(length=20) private String phone;
0014  @Column(length=500) private String resume;
0015  @Column(nullable=false) private Long jobOpeningId;
0016  @Column(nullable=false,length=30) private String applicationStatus = "APPLIED";
0017  private LocalDate appliedAt = LocalDate.now();
0018
0019  public String getStatus() { return applicationStatus; }
0020  public void setStatus(String status) { this.applicationStatus = status; }
0021  public String getCandidateName() { return name; }
0022 }
```

src/main/java/com/hrgenius/entity/Department.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.time.*;
0005 import java.math.BigDecimal;
0006 @Entity
0007 @Table(name="department")
0008 @Getter @Setter @NoArgsConstructor
0009 public class Department {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(nullable=false, unique=true, length=100) private String name; @Column(length=500) private String description;
0012 }
```

src/main/java/com/hrgenius/entity/Employee.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.time.*;
0005 import java.math.BigDecimal;
0006 @Entity
0007 @Table(name="employee")
0008 @Getter @Setter @NoArgsConstructor
0009 public class Employee {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(nullable=false,unique=true,length=20) private String employeeCode; @Column(nullable=false,length=150) private String name; @Column(nullable=false,unique=true,length=150) pri
vate String email; @Column(length=20) private String phone; @Column(nullable=false) private LocalDate dateOfJoining; @Column(nullable=false,length=100) private String designation; @Column
(nullable=false,length=20) private String status; @Column(nullable=false) private Long departmentId; private Long managerId;
0012 }
```

src/main/java/com/hrgenius/entity/Goal.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
```

```
0004 import java.time.LocalDate;
0005
0006 @Entity
0007 @Table(name="goal")
0008 @Getter @Setter @NoArgsConstructor
0009 public class Goal {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(nullable=false) private Long employeeId;
0012  private Long cycleId;
0013  @Column(nullable=false,length=150) private String title;
0014  @Column(columnDefinition="TEXT") private String description;
0015  @Column(nullable=false,length=30) private String status = "IN_PROGRESS";
0016  private Long setBy;
0017  private LocalDate dueDate;
0018 }
```

src/main/java/com/hrgenius/entity/Interview.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.time.*;
0005
0006 @Entity
0007 @Table(name="interview")
0008 @Getter @Setter @NoArgsConstructor
0009 public class Interview {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(nullable=false) private Long candidateId;
0012  @Column(nullable=false) private Long interviewerId;
0013  @Column(nullable=false) private LocalDateTime scheduledDate = LocalDateTime.now();
0014  @Column(columnDefinition="TEXT") private String feedback;
0015  @Column(length=30) private String result = "SCHEDULED";
0016  @Column(length=50) private String mode = "Virtual";
0017  @Column(length=255) private String meetingLink;
0018
0019  public LocalDateTime getScheduledAt() { return scheduledDate; }
0020  public void setScheduledAt(LocalDateTime scheduledAt) { this.scheduledDate = scheduledAt; }
0021  public String getStatus() { return result; }
0022  public void setStatus(String status) { this.result = status; }
0023 }
```

src/main/java/com/hrgenius/entity/JobOpening.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.time.*;
0005
0006 @Entity
0007 @Table(name="jobOpening")
0008 @Getter @Setter @NoArgsConstructor
0009 public class JobOpening {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(nullable=false,length=150) private String title;
0012  private Long departmentId;
0013  @Column(columnDefinition="TEXT") private String description;
0014  @Column(nullable=false) private Integer vacancies = 1;
0015  @Column(nullable=false,length=20) private String status = "OPEN";
0016  @Column(length=100) private String location = "Hybrid";
0017  @Column(nullable=false) private LocalDate postedDate = LocalDate.now();
0018 }
```

src/main/java/com/hrgenius/entity/OnboardingTask.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.time.*;
0005
0006 @Entity
0007 @Table(name="onboardingTask")
0008 @Getter @Setter @NoArgsConstructor
0009 public class OnboardingTask {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(nullable=false) private Long employeeId;
0012  @Column(nullable=false,length=150) private String taskName;
0013  @Column(length=500) private String description;
0014  @Column(nullable=false,length=30) private String status = "PENDING";
0015  private LocalDate dueDate;
0016
0017  public String getTitle() { return taskName; }
0018  public void setTitle(String title) { this.taskName = title; }
0019 }
```

src/main/java/com/hrgenius/entity/Payroll.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.math.BigDecimal;
0005
0006 @Entity
0007 @Table(name="payroll", uniqueConstraints=@UniqueConstraint(columnNames={"employee_id","payroll_month","payroll_year"}))
0008 @Getter @Setter @NoArgsConstructor
0009 public class Payroll {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(name="employee_id", nullable=false) private Long employeeId;
0012  @Column(name="payroll_month", nullable=false) private Integer month;
0013  @Column(name="payroll_year", nullable=false) private Integer year;
0014  @Column(nullable=false,precision=12,scale=2) private BigDecimal grossPay;
0015  @Column(nullable=false,precision=12,scale=2) private BigDecimal pf;
0016  @Column(nullable=false,precision=12,scale=2) private BigDecimal tds;
0017  @Column(nullable=false,precision=12,scale=2) private BigDecimal otherDeductions;
0018  @Column(nullable=false,precision=12,scale=2) private BigDecimal totalDeductions;
0019  @Column(nullable=false,precision=12,scale=2) private BigDecimal netPay;
0020  @Column(nullable=false,length=20) private String status = "PROCESSED";
0021
0022  public BigDecimal getGrossSalary() { return grossPay; }
0023 }
```

src/main/java/com/hrgenius/entity/PerformanceCycle.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.time.*;
0005
0006 @Entity
0007 @Table(name="performanceCycle")
0008 @Getter @Setter @NoArgsConstructor
0009 public class PerformanceCycle {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(nullable=false,length=100) private String name;
0012  @Column(nullable=false) private LocalDate startDate;
0013  @Column(nullable=false) private LocalDate endDate;
```

```
0014  @Column(length=20) private String status = "ACTIVE";
0015 }
```

src/main/java/com/hrgenius/entity/PerformanceReview.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004
0005 @Entity
0006 @Table(name="performanceReview")
0007 @Getter @Setter @NoArgsConstructor
0008 public class PerformanceReview {
0009  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0010  @Column(nullable=false) private Long employeeId;
0011  private Long reviewerId;
0012  @Column(nullable=false) private Long cycleId;
0013  @Column(columnDefinition="TEXT") private String goals;
0014  private Integer rating;
0015  @Column(columnDefinition="TEXT") private String comments;
0016  @Column(columnDefinition="TEXT") private String selfAssessment;
0017  @Column(length=20) private String status = "SUBMITTED";
0018 }
```

src/main/java/com/hrgenius/entity/SalaryStructure.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.math.BigDecimal;
0005
0006 @Entity
0007 @Table(name="salaryStructure")
0008 @Getter @Setter @NoArgsConstructor
0009 public class SalaryStructure {
0010  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0011  @Column(nullable=false,unique=true) private Long employeeId;
0012  @Column(nullable=false,precision=12,scale=2) private BigDecimal basicPay = BigDecimal.ZERO;
0013  @Column(nullable=false,precision=12,scale=2) private BigDecimal hra = BigDecimal.ZERO;
0014  @Column(nullable=false,precision=12,scale=2) private BigDecimal allowances = BigDecimal.ZERO;
0015  @Column(nullable=false,precision=12,scale=2) private BigDecimal deductions = BigDecimal.ZERO;
0016
0017  public BigDecimal getBasic() { return basicPay; }
0018  public void setBasic(BigDecimal b) { this.basicPay = b; }
0019  public BigDecimal getOtherAllowances() { return allowances; }
0020  public void setOtherAllowances(BigDecimal a) { this.allowances = a; }
0021 }
```

src/main/java/com/hrgenius/entity/User.java

```
0001 package com.hrgenius.entity;
0002 import jakarta.persistence.*;
0003 import lombok.*;
0004 import java.time.*;
0005 import java.math.BigDecimal;
0006
0007 @Entity
0008 @Table(name="users")
0009 @Getter @Setter @NoArgsConstructor
0010 public class User {
0011  @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
0012  @Column(nullable=false,unique=true,length=100) private String username;
0013  @Column(nullable=false,length=255) private String password;
```

```
0014  @Column(nullable=false,length=30) private String role;
0015  @Column(nullable=false,length=20) private String status;
0016  private Long employeeId;
0017  private Long candidateId;
0018 }
```

src/main/java/com/hrgenius/exception/ApiExceptionHandler.java

```
0001 package com.hrgenius.exception;
0002 import org.springframework.http.*; import org.springframework.web.bind.MethodArgumentNotValidException; import org.springframework.web.bind.annotation.*;
0003 import java.util.*;
0004 @RestControllerAdvice
0005 public class ApiExceptionHandler {
0006  @ExceptionHandler(NoSuchElementException.class) ResponseEntity<?> missing(Exception e){return ResponseEntity.status(404).body(Map.of("error","NOT_FOUND","message",e.getMessage()));}
0007  @ExceptionHandler(IllegalArgumentException.class) ResponseEntity<?> bad(Exception e){return ResponseEntity.badRequest().body(Map.of("error","BAD_REQUEST","message",e.getMessage()));
}
0008  @ExceptionHandler(MethodArgumentNotValidException.class) ResponseEntity<?> invalid(MethodArgumentNotValidException e){return ResponseEntity.badRequest().body(Map.of("error","VALIDAT
ION_ERROR","message","Request validation failed"));}
0009  @ExceptionHandler(Exception.class) ResponseEntity<?> internal(Exception e){return ResponseEntity.status(500).body(Map.of("error","INTERNAL_ERROR","message","An unexpected error occu
rred"));}
0010 }
```

src/main/java/com/hrgenius/repository/CandidateRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.Candidate;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005
0006 public interface CandidateRepository extends JpaRepository<Candidate, Long> {
0007  List<Candidate> findByEmail(String email);
0008  List<Candidate> findByJobOpeningId(Long jobOpeningId);
0009  List<Candidate> findByApplicationStatus(String applicationStatus);
0010  List<Candidate> findByJobOpeningIdAndApplicationStatus(Long jobOpeningId, String applicationStatus);
0011 }
```

src/main/java/com/hrgenius/repository/DepartmentRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.Department;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005 public interface DepartmentRepository extends JpaRepository<Department, Long> {
0006
0007
0008
0009
0010 }
```

src/main/java/com/hrgenius/repository/EmployeeRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.Employee;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005 import org.springframework.data.domain.Page;
0006
0007 public interface EmployeeRepository extends JpaRepository<Employee, Long> {
0008  Page<Employee> findByNameContainingIgnoreCaseOrEmailContainingIgnoreCase(String name, String email, org.springframework.data.domain.Pageable pageable);
0009  Optional<Employee> findByEmail(String email);
0010  List<Employee> findByDepartmentId(Long departmentId);
0011  List<Employee> findByManagerId(Long managerId);
0012 }
```

src/main/java/com/hrgenius/repository/GoalRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.Goal;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005
0006 public interface GoalRepository extends JpaRepository<Goal, Long> {
0007  List<Goal> findByEmployeeId(Long employeeId);
0008  List<Goal> findByCycleId(Long cycleId);
0009 }
```

src/main/java/com/hrgenius/repository/InterviewRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.Interview;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005
0006 public interface InterviewRepository extends JpaRepository<Interview, Long> {
0007  List<Interview> findByCandidateId(Long candidateId);
0008  List<Interview> findByInterviewerId(Long interviewerId);
0009 }
```

src/main/java/com/hrgenius/repository/JobOpeningRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.JobOpening;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005 public interface JobOpeningRepository extends JpaRepository<JobOpening, Long> {
0006
0007
0008
0009
0010 }
```

src/main/java/com/hrgenius/repository/OnboardingTaskRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.OnboardingTask;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005
0006 public interface OnboardingTaskRepository extends JpaRepository<OnboardingTask, Long> {
0007  List<OnboardingTask> findByEmployeeId(Long employeeId);
0008 }
```

src/main/java/com/hrgenius/repository/PayrollRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.Payroll;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005
0006 public interface PayrollRepository extends JpaRepository<Payroll, Long> {
0007  boolean existsByEmployeeIdAndMonthAndYear(Long employeeId, Integer month, Integer year);
0008  List<Payroll> findByEmployeeIdOrderByYearDescMonthDesc(Long employeeId);
0009  Optional<Payroll> findByEmployeeIdAndMonthAndYear(Long employeeId, Integer month, Integer year);
0010 }
```

src/main/java/com/hrgenius/repository/PerformanceCycleRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.PerformanceCycle;
```

```
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005 public interface PerformanceCycleRepository extends JpaRepository<PerformanceCycle, Long> {
0006
0007
0008
0009
0010 }
```

src/main/java/com/hrgenius/repository/PerformanceReviewRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.PerformanceReview;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005
0006 public interface PerformanceReviewRepository extends JpaRepository<PerformanceReview, Long> {
0007  List<PerformanceReview> findByEmployeeId(Long employeeId);
0008  List<PerformanceReview> findByCycleId(Long cycleId);
0009 }
```

src/main/java/com/hrgenius/repository/SalaryStructureRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.SalaryStructure;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005 public interface SalaryStructureRepository extends JpaRepository<SalaryStructure, Long> {
0006
0007
0008  Optional<SalaryStructure> findByEmployeeId(Long employeeId);
0009
0010 }
```

src/main/java/com/hrgenius/repository/UserRepository.java

```
0001 package com.hrgenius.repository;
0002 import com.hrgenius.entity.User;
0003 import org.springframework.data.jpa.repository.JpaRepository;
0004 import java.util.*;
0005 public interface UserRepository extends JpaRepository<User, Long> {
0006  Optional<User> findByUsername(String username);
0007
0008
0009
0010 }
```

src/main/java/com/hrgenius/security/JwtFilter.java

```
0001 package com.hrgenius.security;
0002 import io.jsonwebtoken.Claims;
0003 import jakarta.servlet.*; import jakarta.servlet.http.*;
0004 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
0005 import org.springframework.security.core.authority.SimpleGrantedAuthority;
0006 import org.springframework.security.core.context.SecurityContextHolder;
0007 import org.springframework.stereotype.Component; import org.springframework.web.filter.OncePerRequestFilter;
0008 import java.io.IOException; import java.util.List;
0009 @Component
0010 public class JwtFilter extends OncePerRequestFilter {
0011  private final JwtService jwt; public JwtFilter(JwtService jwt){this.jwt=jwt;}
0012  @Override protected void doFilterInternal(HttpServletRequest req,HttpServletResponse res,FilterChain chain)throws ServletException,IOException {
0013   String h=req.getHeader("Authorization");
0014   if(h!=null && h.startsWith("Bearer ") && SecurityContextHolder.getContext().getAuthentication()==null) {
0015    try { Claims c=jwt.parse(h.substring(7)); String role=c.get("role",String.class);
```

```
0016     var auth=new UsernamePasswordAuthenticationToken(c.getSubject(),null,List.of(new SimpleGrantedAuthority("ROLE_"+role)));
0017     SecurityContextHolder.getContext().setAuthentication(auth);
0018    } catch(Exception ignored) { /* invalid tokens remain unauthenticated */ }
0019   }
0020   chain.doFilter(req,res);
0021  }
0022 }
```

src/main/java/com/hrgenius/security/JwtService.java

```
0001 package com.hrgenius.security;
0002 import io.jsonwebtoken.*;
0003 import io.jsonwebtoken.security.Keys;
0004 import org.springframework.beans.factory.annotation.Value;
0005 import org.springframework.stereotype.Service;
0006 import javax.crypto.SecretKey;
0007 import java.nio.charset.StandardCharsets;
0008 import java.util.Date;
0009 @Service
0010 public class JwtService {
0011  private final SecretKey key; private final long expiry;
0012  public JwtService(@Value("${app.jwt.secret}") String secret,@Value("${app.jwt.expiration-ms}") long expiry) {
0013   this.key=Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8)); this.expiry=expiry;
0014  }
0015  public String generate(String username,String role) {
0016   long now=System.currentTimeMillis();
0017   return Jwts.builder().subject(username).claim("role",role).issuedAt(new Date(now)).expiration(new Date(now+expiry)).signWith(key).compact();
0018  }
0019  public Claims parse(String token) { return Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload(); }
0020 }
```

src/main/java/com/hrgenius/service/AuthService.java

```
0001 package com.hrgenius.service;
0002 import com.hrgenius.entity.User;
0003 import com.hrgenius.repository.UserRepository;
0004 import com.hrgenius.security.JwtService;
0005 import org.springframework.security.crypto.password.PasswordEncoder;
0006 import org.springframework.stereotype.Service;
0007 import java.util.Map;
0008
0009 @Service
0010 public class AuthService {
0011  private final UserRepository users;
0012  private final PasswordEncoder encoder;
0013  private final JwtService jwt;
0014
0015  public AuthService(UserRepository users, PasswordEncoder encoder, JwtService jwt) {
0016   this.users = users;
0017   this.encoder = encoder;
0018   this.jwt = jwt;
0019  }
0020
0021  public Map<String,Object> login(String username, String password) {
0022   User u = users.findByUsername(username).orElseThrow(() -> new IllegalArgumentException("Invalid credentials"));
0023   if (!"ACTIVE".equals(u.getStatus()) || !encoder.matches(password, u.getPassword())) {
0024    throw new IllegalArgumentException("Invalid credentials");
0025   }
0026   return Map.of("token", jwt.generate(u.getUsername(), u.getRole()), "tokenType", "Bearer", "role", u.getRole(), "username", u.getUsername());
0027  }
0028
0029  public void registerCandidate(String username, String password) {
0030   if (users.findByUsername(username).isPresent()) throw new IllegalArgumentException("Username already exists");
```

```
0031   User u = new User();
0032   u.setUsername(username);
0033   u.setPassword(encoder.encode(password));
0034   u.setRole("CANDIDATE");
0035   u.setStatus("ACTIVE");
0036   users.save(u);
0037  }
0038
0039  public void changePassword(String username, String currentPassword, String newPassword) {
0040   User u = users.findByUsername(username).orElseThrow(() -> new IllegalArgumentException("User not found"));
0041   if (!encoder.matches(currentPassword, u.getPassword())) {
0042    throw new IllegalArgumentException("Current password is incorrect");
0043   }
0044   u.setPassword(encoder.encode(newPassword));
0045   users.save(u);
0046  }
0047 }
```

src/main/java/com/hrgenius/service/EmployeeService.java

```
0001 package com.hrgenius.service;
0002
0003 import com.hrgenius.dto.EmployeeRequest;
0004 import com.hrgenius.dto.EmployeeResponse;
0005 import com.hrgenius.entity.Department;
0006 import com.hrgenius.entity.Employee;
0007 import com.hrgenius.repository.DepartmentRepository;
0008 import com.hrgenius.repository.EmployeeRepository;
0009 import org.springframework.data.domain.*;
0010 import org.springframework.stereotype.Service;
0011
0012 import java.util.*;
0013
0014 @Service
0015 public class EmployeeService {
0016  private final EmployeeRepository repo;
0017  private final DepartmentRepository deptRepo;
0018
0019  public EmployeeService(EmployeeRepository repo, DepartmentRepository deptRepo) {
0020   this.repo = repo;
0021   this.deptRepo = deptRepo;
0022  }
0023
0024  public Employee create(EmployeeRequest r) {
0025   Employee e = new Employee();
0026   e.setEmployeeCode("EMP-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase());
0027   e.setName(r.name());
0028   e.setEmail(r.email());
0029   e.setPhone(r.phone());
0030   e.setDesignation(r.designation());
0031   e.setDepartmentId(r.departmentId());
0032   e.setDateOfJoining(r.dateOfJoining());
0033   e.setManagerId(r.managerId());
0034   e.setStatus("ACTIVE");
0035   return repo.save(e);
0036  }
0037
0038  public Page<EmployeeResponse> list(String q, Pageable p) {
0039   Map<Long, String> deptMap = getDepartmentNameMap();
0040   Page<Employee> page = (q == null || q.isBlank())
0041       ? repo.findAll(p)
0042       : repo.findByNameContainingIgnoreCaseOrEmailContainingIgnoreCase(q, q, p);
```

```
0043   return page.map(e -> EmployeeResponse.from(e, deptMap.get(e.getDepartmentId())));
0044  }
0045
0046  public Employee get(long id) {
0047   return repo.findById(id).orElseThrow(() -> new java.util.NoSuchElementException("Employee not found"));
0048  }
0049
0050  public EmployeeResponse getResponse(long id) {
0051   Employee e = get(id);
0052   String deptName = e.getDepartmentId() != null
0053       ? deptRepo.findById(e.getDepartmentId()).map(Department::getName).orElse(null)
0054       : null;
0055   return EmployeeResponse.from(e, deptName);
0056  }
0057
0058  public Employee update(long id, EmployeeRequest r) {
0059   Employee e = get(id);
0060   e.setName(r.name());
0061   e.setEmail(r.email());
0062   e.setPhone(r.phone());
0063   e.setDesignation(r.designation());
0064   e.setDepartmentId(r.departmentId());
0065   e.setDateOfJoining(r.dateOfJoining());
0066   e.setManagerId(r.managerId());
0067   return repo.save(e);
0068  }
0069
0070  public void deactivate(long id) {
0071   Employee e = get(id);
0072   e.setStatus("INACTIVE");
0073   repo.save(e);
0074  }
0075
0076  public EmployeeResponse getForUsername(String username) {
0077   Employee emp = repo.findByEmail(username)
0078       .or(() -> repo.findAll().stream().filter(e -> username.equalsIgnoreCase(e.getEmail())).findFirst())
0079       .orElseGet(() -> repo.findAll().stream().findFirst().orElseGet(() -> {
0080        Employee fallback = new Employee();
0081        fallback.setId(1L);
0082        fallback.setEmployeeCode("EMP-ADMIN");
0083        fallback.setName(username);
0084        fallback.setEmail(username + "@hrgenius.local");
0085        fallback.setDesignation("Administrator");
0086        fallback.setStatus("ACTIVE");
0087        return fallback;
0088       }));
0089   String deptName = emp.getDepartmentId() != null
0090       ? deptRepo.findById(emp.getDepartmentId()).map(Department::getName).orElse(null)
0091       : null;
0092   return EmployeeResponse.from(emp, deptName);
0093  }
0094
0095  public List<EmployeeResponse> getTeam(long id) {
0096   Map<Long, String> deptMap = getDepartmentNameMap();
0097   Employee emp = get(id);
0098   List<Employee> reports = repo.findByManagerId(id);
0099   if (!reports.isEmpty()) {
0100    return reports.stream().map(e -> EmployeeResponse.from(e, deptMap.get(e.getDepartmentId()))).toList();
0101   }
0102   if (emp.getDepartmentId() != null) {
0103    return repo.findByDepartmentId(emp.getDepartmentId()).stream()
```

```
0104        .filter(e -> !e.getId().equals(id))
0105        .map(e -> EmployeeResponse.from(e, deptMap.get(e.getDepartmentId())))
0106        .toList();
0107    }
0108    return List.of();
0109   }
0110
0111   private Map<Long, String> getDepartmentNameMap() {
0112    Map<Long, String> map = new HashMap<>();
0113    for (Department d : deptRepo.findAll()) {
0114     map.put(d.getId(), d.getName());
0115    }
0116    return map;
0117   }
0118 }
```

src/main/java/com/hrgenius/service/PayrollCalculator.java

```
0001 package com.hrgenius.service;
0002 import java.math.*;
0003 /** Centralises the specified payroll formulas and HALF_UP monetary rounding. */
0004 public final class PayrollCalculator {
0005   private static final BigDecimal HUNDRED=new BigDecimal("100");
0006   private PayrollCalculator(){}
0007   public record Result(BigDecimal gross,BigDecimal pf,BigDecimal tds,BigDecimal otherDeduction,BigDecimal totalDeductions,BigDecimal net){}
0008   private static BigDecimal money(BigDecimal n){return n.setScale(2,RoundingMode.HALF_UP);}
0009   public static BigDecimal monthlyTds(BigDecimal monthlyGross){
0010    BigDecimal annual=monthlyGross.multiply(BigDecimal.valueOf(12));
0011    BigDecimal rate=annual.compareTo(new BigDecimal("300000"))<=0?BigDecimal.ZERO:
0012     annual.compareTo(new BigDecimal("600000"))<=0?new BigDecimal("0.05"):
0013     annual.compareTo(new BigDecimal("1000000"))<=0?new BigDecimal("0.10"):new BigDecimal("0.15");
0014    return money(monthlyGross.multiply(rate));
0015   }
0016   public static Result calculate(BigDecimal basic,BigDecimal hra,BigDecimal allowances,BigDecimal otherDeduction,BigDecimal tds){
0017    BigDecimal gross=money(basic.add(hra).add(allowances)); BigDecimal pf=money(basic.multiply(new BigDecimal("0.12")));
0018    BigDecimal tax=money(tds); BigDecimal other=money(otherDeduction);
0019    BigDecimal total=money(pf.add(tax).add(other)); return new Result(gross,pf,tax,other,total,money(gross.subtract(total)));
0020   }
0021 }
```

src/test/java/com/hrgenius/PayrollCalculatorTest.java

```
0001 package com.hrgenius;
0002 import com.hrgenius.service.PayrollCalculator; import org.junit.jupiter.api.Test; import java.math.BigDecimal;
0003 import static org.junit.jupiter.api.Assertions.*;
0004 class PayrollCalculatorTest {
0005   @Test void reproducesDocumentedFirstExample(){var r=PayrollCalculator.calculate(new BigDecimal("50000"),new BigDecimal("20000"),new BigDecimal("5000"),new BigDecimal("200"),new BigDecimal("625"));assertEquals(new BigDecimal("68175.00"),r.net());}
0006   @Test void reproducesDocumentedSecondExample(){var r=PayrollCalculator.calculate(new BigDecimal("30000"),new BigDecimal("12000"),new BigDecimal("3000"),new BigDecimal("200"),BigDecimal.ZERO);assertEquals(new BigDecimal("41200.00"),r.net());}
0007 }
```

02. FRONTEND CODE

src/main/resources/static/app.js

```
0001 /* HRGenius frontend. All API-provided values are escaped before HTML insertion. */
0002 (() => {
0003   'use strict';
0004   const API = '/api/v1';
0005   const $ = (id) => document.getElementById(id);
0006   const state = { token: null, role: null, username: null, page: 'dashboard', pageNo: 0, pageSize: 10, employeeQuery: '', charts: [], departments: [], jobs: [], employees: [] };
0007   const roles = {
0008     admin: ['dashboard','employees','departments','jobs','candidates','interviews','onboarding','salary','payroll','performance','analytics','password'],
0009     hr_manager: ['dashboard','employees','departments','jobs','candidates','interviews','onboarding','salary','payroll','performance','analytics','password'],
0010     manager: ['dashboard','employees','performance','password'],
0011     employee: ['dashboard','onboarding','salary','payroll','performance','password'],
0012     candidate: ['dashboard','jobs','applications','password']
0013   };
0014   const pages = {
0015     dashboard:['Overview','Your HR workspace'], employees:['Employees','Manage employee records and team information'], departments:['Departments','Organise teams and reporting struc
tures'], jobs:['Job openings','Explore open roles and manage recruitment'], candidates:['Candidates','Review applications and candidate progress'], interviews:['Interviews','Schedule inte
rviews and capture feedback'], onboarding:['Onboarding','Track employee onboarding activities'], salary:['Salary structures','Maintain employee compensation components'], payroll:['Payrol
l','Run payroll and access payslips'], performance:['Performance','Cycles, goals and employee reviews'], analytics:['HR analytics','Workforce insights and operational metrics'], applicati
ons:['My applications','Track the status of your job applications'], password:['Change password','Keep your account secure']
0016   };
0017   const nav = [
0018     ['dashboard','\u2302','Overview'],['employees','\u2659','Employees'],['departments','\u25a6','Departments'],['jobs','\u2315','Job openings'],['candidates','\u2667','Candidates'],
['interviews','\u25f7','Interviews'],['onboarding','\u2713','Onboarding'],['salary','\u20b9','Salary structures'],['payroll','\u25a4','Payroll'],['performance','\u2727','Performance'],['a
nalytics','\u25a5','Analytics'],['applications','\u25a3','My applications'],['password','\u2699','Change password']
0019   ];
0020   const esc = (v) => String(v ?? '').replace(/[&<>"']/g, c => ({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[c]));
0021   const fmt = (v) => v == null || v === '' ? '\u2014' : esc(v);
0022   const money = (v) => v == null ? '\u2014' : '\u20b9' + Number(v).toLocaleString('en-IN',{maximumFractionDigits:2});
0023   const arr = (v) => Array.isArray(v) ? v : (v?.content || v?.data || []);
0024   const role = () => (state.role || '').toLowerCase().replace(/^role_/, '');
0025   const can = (...allowed) => allowed.map(x=>x.toLowerCase()).includes(role());
0026   const toast = (message, error=false) => { const el=document.createElement('div'); el.className='toast'+(error?' error':''); el.textContent=message; $('toastRegion').append(el); set
Timeout(()=>el.remove(),4200); };
0027   async function api(path, options={}) {
0028     const headers = {...(options.headers||{})};
0029     if (options.body && !(options.body instanceof FormData)) headers['Content-Type']='application/json';
0030     if (state.token) headers.Authorization='Bearer '+state.token;
0031     let response;
0032     try { response=await fetch(API+path,{...options,headers}); }
0033     catch { throw new Error('Unable to reach the HRGenius server. Check that the application is running and try again.'); }
0034     if (!response.ok) {
0035       let message=`Request failed (${response.status})`;
0036       try { const body=await response.json(); message=body.message || body.error || body.detail || message; if(body.errors) message += ': '+Object.values(body.errors).join(', '); } c
atch { try { message=(await response.text())||message; } catch {} }
0037       if(response.status===401){ state.token=null; state.role=null; showAuth(); }
0038       throw new Error(message);
0039     }
0040     if(response.status===204) return null;
0041     const type=response.headers.get('content-type')||'';
0042     if(type.includes('application/pdf') || type.includes('application/octet-stream')) return response.blob();
0043     const text=await response.text(); if(!text) return null;
0044     try{return JSON.parse(text);}catch{return text;}}
0045   const get = (path) => api(path);
0046   const post = (path, body) => api(path,{method:'POST',body:JSON.stringify(body)});
0047   const put = (path, body) => api(path,{method:'PUT',body:JSON.stringify(body)});
```

```
0048   const del = (path) => api(path,{method:'DELETE'});
0049   function showAuth(mode='login') {
0050     $('appShell').hidden=true;
0051     $('appShell').style.display='none';
0052     $('authScreen').hidden=false;
0053     $('authScreen').style.display='grid';
0054     $('loginPane').hidden=mode!=='login';
0055     $('registerPane').hidden=mode!=='register';
0056     $('loginError').textContent='';
0057     $('registerError').textContent='';
0058   }
0059   function showApp(user) {
0060     $('authScreen').hidden=true;
0061     $('authScreen').style.display='none';
0062     $('appShell').hidden=false;
0063     $('appShell').style.display='block';
0064     $('userName').textContent=user || 'User'; $('userRole').textContent=(state.role||'').toUpperCase(); $('userAvatar').textContent=(user||'U').trim().charAt(0).toUpperCase();
0065     $('todayLabel').textContent=new Intl.DateTimeFormat('en-IN',{day:'2-digit',month:'short',year:'numeric'}).format(new Date());
0066     buildNav(); navigate('dashboard');
0067   }
0068   function buildNav(){
0069     const allowed=roles[role()]||['dashboard','password'];
0070     $('sideNav').innerHTML=nav.filter(n=>allowed.includes(n[0])).map(n=>`<button class="nav-item ${state.page===n[0]?'active':''}" data-page="${n[0]}"><span class="nav-icon">${n[1]}<
/span>${n[2]}</button>`).join('');
0071     $('sideNav').querySelectorAll('[data-page]').forEach(b=>b.addEventListener('click',()=>navigate(b.dataset.page)));
0072   }
0073   function navigate(page){
0074     const allowed=roles[role()]||[]; if(!allowed.includes(page)){toast('Your role does not have access to this page.',true);return;}
0075     state.page=page;state.pageNo=0; buildNav(); $('sidebar').classList.remove('open');
0076     $('pageCrumb').textContent=pages[page]?.[0]||'Overview';
0077     $('pageContent').innerHTML=`<div class="page-heading"><div><span class="eyebrow eyebrow-dark">HRGENIUS WORKSPACE</span><h1>${esc(pages[page]?.[0]||'Overview')}</h1><p>${esc(pages
[page]?.[1]||'')}</p></div><div class="heading-actions" id="pageActions"></div></div><div id="pageBody"><div class="card"><div class="empty-state"><div class="empty-icon">\u25cc</div><str
ong>Loading workspace</strong><p>Connecting to your HR data\u2026</p></div></div></div>`;
0078     const renderers={dashboard:renderDashboard,employees:renderEmployees,departments:renderDepartments,jobs:renderJobs,candidates:renderCandidates,interviews:renderInterviews,onboard
ing:renderOnboarding,salary:renderSalary,payroll:renderPayroll,performance:renderPerformance,analytics:renderAnalytics,applications:renderApplications,password:renderPassword};
0079     (renderers[page]||renderDashboard)().catch(e=>showError(e.message));
0080   }
0081   function showError(message){$('pageBody').innerHTML=`<div class="error-panel">${esc(message)}</div>`;}
0082   function headingAction(html){$('pageActions').innerHTML=html;}
0083   function empty(title,desc){return `<div class="empty-state"><div class="empty-icon">\u2301</div><strong>${esc(title)}</strong><p>${esc(desc)}</p></div>`;}
0084   function card(title,body,subtitle=''){return `<section class="card card-pad"><div class="card-title"><div><h3>${esc(title)}</h3>${subtitle?`<p>${esc(subtitle)}</p>`:''}</div></div>
${body}</section>`;}
0085   function status(s){const c=String(s||'unknown').toLowerCase().replace(/[^a-z-]/g,'-');return `<span class="status ${c}">${esc(s||'Unknown')}</span>`;}
0086   function table(headers,rows,emptyText='No records found.'){return `<div class="table-wrap"><table><thead><tr>${headers.map(h=>`<th>${esc(h)}</th>`).join('')}</tr></thead><tbody>${r
ows||''}</tbody></table></div>${rows?'':empty('Nothing to display',emptyText)}`;}
0087   function field(name,label,type='text',required=false,placeholder='',extra=''){return `<div class="field"><label for="f_${esc(name)}">${esc(label)}${required?' *':''}</label><input
id="f_${esc(name)}" name="${esc(name)}" type="${esc(type)}" ${required?'required':''} placeholder="${esc(placeholder)}" ${extra}></div>`;}
0088   function selectField(name,label,options,required=true){return `<div class="field"><label for="f_${esc(name)}">${esc(label)}${required?' *':''}</label><select id="f_${esc(name)}" na
me="${esc(name)}" ${required?'required':''}><option value="">Select ${esc(label.toLowerCase())}</option>${options.map(o=>`<option value="${esc(o.value)}">${esc(o.label)}</option>`).join('
')}</select></div>`;}
0089   function formModal(title,fields,submitLabel,handler){
0090     const root=document.createElement('div');root.className='modal-backdrop';root.innerHTML=`<div class="modal" role="dialog" aria-modal="true"><div class="modal-head"><h3>${esc(titl
e)}</h3><button class="icon-button" type="button" data-close aria-label="Close">\u00d7</button></div><form id="modalForm"><div class="form-grid">${fields}</div><p class="form-error" id="m
odalError"></p><div class="form-actions"><button type="button" class="btn btn-outline" data-close>Cancel</button><button class="btn btn-primary" type="submit">${esc(submitLabel)}</button>
</div></form></div>`;
0091     document.body.append(root);root.querySelectorAll('[data-close]').forEach(b=>b.onclick=()=>root.remove());root.addEventListener('click',e=>{if(e.target===root)root.remove();});roo
t.querySelector('form').onsubmit=async e=>{e.preventDefault();const data=Object.fromEntries(new FormData(e.target));const button=e.target.querySelector('[type=submit]');button.disabled=tr
ue;try{await handler(data);root.remove();toast('Changes saved successfully.');await navigate(state.page);}catch(err){root.querySelector('#modalError').textContent=err.message;button.disab
led=false;}};
0092   }
0093   async function loadEmployeesList(){try{const d=await get('/employees?page=0&size=200');state.employees=arr(d);}catch{state.employees=[];}return state.employees;}
```

```
0094   async function renderDashboard(){
0095     const isHr=can('ADMIN','HR_MANAGER'); let metrics=[];
0096     if(isHr){const paths=[['Headcount','/analytics/headcount','\u2659'],['Hiring','/analytics/hiring','\u2315'],['Attrition','/analytics/attrition','\u2198'],['Payroll cost','/analyt
ics/payroll-cost','\u20b9']];metrics=await Promise.all(paths.map(async p=>{try{const d=await get(p[1]);return {label:p[0],value:extractMetric(d),icon:p[2]};}catch{return {label:p[0],value
:'\u2014',icon:p[2]};}}));}
0097     $('pageBody').innerHTML=`${isHr?`<div class="stats-grid">${metrics.map(m=>`<div class="card stat-card"><span class="stat-label">${esc(m.label)}</span><span class="stat-icon">${m.
icon}</span><div class="stat-value">${esc(m.value)}</div><div class="stat-note">Live analytics metric</div></div>`).join('')}</div>`:''}<div class="dashboard-grid">${card('Welcome to HRGe
nius',`<div class="notice">You are signed in as <strong>${esc(state.username||'')}</strong> with the <strong>${esc((state.role||'').replace('ROLE_',''))}</strong> role. Use the navigation
 to access the tools available to your account.</div><div class="detail-row"><span>Workspace</span><strong>HR Management System</strong></div><div class="detail-row"><span>Access profile<
/span><strong>${esc(state.role||'')}</strong></div><div class="detail-row"><span>Session</span>${status('Active')}</div>`,'Your account and workspace')} ${card('Quick access',`<div class=
"quick-links">${(roles[role()]||[]).filter(p=>p!=='dashboard').slice(0,6).map(p=>`<button class="btn btn-secondary" data-quick="${p}">${esc(pages[p][0])} \u2192</button>`).join('')}</div>
`,'Jump to an available module')}</div><div class="dashboard-bottom">${card('Workspace guidance',empty('Your workspace is ready','Use the module navigation to view or manage records permi
tted by your role.'))}${card('Security reminder',`<div class="notice">Never share your password or authentication token. Sign out when using a shared device.</div><div class="form-actions
"><button class="btn btn-outline btn-small" data-quick="password">Change password</button></div>`)}</div>`;
0098     $('pageBody').querySelectorAll('[data-quick]').forEach(b=>b.onclick=()=>navigate(b.dataset.quick));
0099   }
0100   function extractMetric(d){if(d==null)return '\u2014';if(typeof d==='number'||typeof d==='string')return d;if(Array.isArray(d))return d.length;for(const k of ['totalCost','headcount
','totalHires','totalAttrition','totalReviews','total','count','value'])if(d[k]!=null)return d[k];return 'Available';}
0101   async function renderEmployees(){
0102     headingAction(can('ADMIN','HR_MANAGER')?'<button class="btn btn-primary" id="addEmployee">\uff0b Add employee</button>':'');
0103     const q=state.employeeQuery;const d=await get(`/employees?page=${state.pageNo}&size=${state.pageSize}&sort=name,asc${q?'&search='+encodeURIComponent(q):''}`);const rows=arr(d).ma
p(e=>`<tr><td><strong>${fmt(e.employeeCode)}</strong></td><td><div class="person-cell"><span class="mini-avatar">${esc((e.name||'E').charAt(0).toUpperCase())}</span><span><strong>${fmt(e.
name)}</strong><small>${fmt(e.email)}</small></span></div></td><td>${fmt(e.designation)}</td><td>${fmt(e.departmentName||e.department?.name)}</td><td>${status(e.status)}</td><td>${can('AD
MIN','HR_MANAGER')?`<button class="btn btn-outline btn-small" data-edit="${esc(e.id)}">Edit</button> <button class="btn btn-danger btn-small" data-deactivate="${esc(e.id)}">Deactivate</bu
tton>`:`<button class="btn btn-outline btn-small" data-view="${esc(e.id)}">View</button> <button class="btn btn-outline btn-small" data-team="${esc(e.id)}">Team</button>`}</td></tr>`).joi
n('');
0104     $('pageBody').innerHTML=`<section class="card card-pad"><div class="toolbar"><input id="employeeSearch" placeholder="Search name, email or code" value="${esc(q)}"><button class="
btn btn-secondary btn-small" id="employeeSearchBtn">Search</button><button class="btn btn-outline btn-small" id="clearSearch">Clear</button></div>${table(['Employee code','Employee','Desi
gnation','Department','Status','Actions'],rows,'Employees will appear here when records are available.')}${pager(d)}</section>`;
0105     $('employeeSearchBtn').onclick=()=>{state.employeeQuery=$('employeeSearch').value.trim();state.pageNo=0;renderEmployees().catch(e=>showError(e.message));};$('clearSearch').onclic
k=()=>{state.employeeQuery='';state.pageNo=0;renderEmployees().catch(e=>showError(e.message));};wirePager(d,renderEmployees);
0106     if($('addEmployee'))$('addEmployee').onclick=async()=>{const deps=await get('/departments');formModal('Add employee',field('name','Full name','text',true)+field('email','Email','
email',true)+field('phone','Phone','tel')+field('designation','Designation','text',true)+selectField('departmentId','Department',arr(deps).map(x=>({value:x.id,label:x.name})))+field('date
OfJoining','Date of joining','date',true),'Create employee',async v=>post('/employees',{...v,departmentId:Number(v.departmentId)}));};
0107     $('pageBody').querySelectorAll('[data-deactivate]').forEach(b=>b.onclick=async()=>{if(confirm('Deactivate this employee?'))try{await del('/employees/'+encodeURIComponent(b.datase
t.deactivate));toast('Employee deactivated.');renderEmployees();}catch(e){toast(e.message,true);}});
0108     $('pageBody').querySelectorAll('[data-edit]').forEach(b=>b.onclick=()=>editEmployee(b.dataset.edit));$('pageBody').querySelectorAll('[data-view]').forEach(b=>b.onclick=()=>viewEm
ployee(b.dataset.view));$('pageBody').querySelectorAll('[data-team]').forEach(b=>b.onclick=()=>viewTeam(b.dataset.team));
0109   }
0110   function pager(d){const page=d?.number??state.pageNo,total=d?.totalPages??1,count=d?.totalElements??arr(d).length;return `<div class="pagination"><span>${count} record(s) \u00b7 Pa
ge ${page+1} of ${Math.max(total,1)}</span><div><button id="prevPage" ${page<=0?'disabled':''}>Previous</button><button id="nextPage" ${page+1>=total?'disabled':''}>Next</button></div></d
iv>`;}
0111   function wirePager(d,fn){$('prevPage')?.addEventListener('click',()=>{state.pageNo=Math.max(0,state.pageNo-1);fn().catch(e=>showError(e.message));});$('nextPage')?.addEventListener
('click',()=>{state.pageNo++;fn().catch(e=>showError(e.message));});}
0112   async function viewTeam(id){try{const d=arr(await get('/employees/'+encodeURIComponent(id)+'/team'));const rows=d.map(e=>`<tr><td>${fmt(e.employeeCode)}</td><td>${fmt(e.name)}</td>
<td>${fmt(e.designation)}</td><td>${status(e.status)}</td></tr>`).join('');formModal('Employee team',table(['Code','Employee','Designation','Status'],rows,'No direct reports are currently
 available.'),'Close',async()=>{});}catch(e){toast(e.message,true);}}
0113   async function editEmployee(id){try{const e=await get('/employees/'+encodeURIComponent(id));formModal('Edit employee',field('name','Full name','text',true,'',`value="${esc(e.name)}
"`)+field('email','Email','email',true,'',`value="${esc(e.email)}"`)+field('phone','Phone','tel',false,'',`value="${esc(e.phone||'')}"`)+field('designation','Designation','text',true,'',`
value="${esc(e.designation)}"`),'Save changes',v=>put('/employees/'+encodeURIComponent(id),v));}catch(err){toast(err.message,true);}}
0114   async function viewEmployee(id){try{const e=await get('/employees/'+encodeURIComponent(id));formModal('Employee details',`<div class="field full"><div class="detail-row"><span>Empl
oyee</span><strong>${fmt(e.name)}</strong></div><div class="detail-row"><span>Code</span><strong>${fmt(e.employeeCode)}</strong></div><div class="detail-row"><span>Email</span><strong>${f
mt(e.email)}</strong></div><div class="detail-row"><span>Phone</span><strong>${fmt(e.phone)}</strong></div><div class="detail-row"><span>Designation</span><strong>${fmt(e.designation)}</s
trong></div><div class="detail-row"><span>Department</span><strong>${fmt(e.departmentName)}</strong></div><div class="detail-row"><span>Status</span>${status(e.status)}</div></div>`,'Clos
e',async()=>{});}catch(err){toast(err.message,true);}}
0115   async function renderDepartments(){
0116     headingAction(can('ADMIN','HR_MANAGER')?'<button class="btn btn-primary" id="addDepartment">\uff0b Add department</button>':'');const data=arr(await get('/departments'));const ro
ws=data.map(d=>`<tr><td><strong>${fmt(d.name)}</strong></td><td>${fmt(d.description)}</td><td>${fmt(d.employeeCount??d.employees?.length??0)}</td><td>${can('ADMIN','HR_MANAGER')?`<button
 class="btn btn-outline btn-small" data-dept-edit="${esc(d.id)}">Edit</button>`:'\u2014'}</td></tr>`).join('');$('pageBody').innerHTML=card('Department directory',table(['Department','Desc
ription','Employees','Actions'],rows,'Departments will appear here when configured.'));if($('addDepartment'))$('addDepartment').onclick=()=>formModal('Create department',field('name','Dep
artment name','text',true)+field('description','Description'),'Create department',v=>post('/departments',v));$('pageBody').querySelectorAll('[data-dept-edit]').forEach(b=>b.onclick=()=>fo
```

```
rmModal('Edit department',field('name','Department name','text',true),'Save department',v=>put('/departments/'+encodeURIComponent(b.dataset.deptEdit),v)));
  }
  async function renderJobs(){
    const isHr=can('ADMIN','HR_MANAGER');headingAction(isHr?'<button class="btn btn-primary" id="addJob">\uff0b Post a job</button>':'');const [data,deps]=await Promise.all([get('/jobs'),get('/departments')]);const jobList=arr(data);state.jobs=jobList;state.departments=arr(deps);
    $('pageBody').innerHTML=`${isHr?`<div class="notice section-gap">Recruitment management is available to HR and administrators. Open roles appear for public candidates to apply.</div>`:''}<div class="job-grid section-gap">${jobList.length?jobList.map(j=>`<article class="card job-card"><span class="eyebrow eyebrow-dark">${fmt(j.departmentName||'OPEN POSITION')}</span><h3>${fmt(j.title)}</h3><div class="job-meta"><span>\u2316 ${fmt(j.location)}</span><span>\u25a6 ${fmt(j.vacancies)} vacancies</span></div><p>${fmt(j.description)}</p><div class="job-card-footer">${status(j.status||'Open')}${can('CANDIDATE')?`<button class="btn btn-primary btn-small" data-apply="${esc(j.id)}">Apply now \u2192</button>`:''}${isHr?`<button class="btn btn-outline btn-small" data-job-edit="${esc(j.id)}">Edit</button>`:''}</div></article>`).join(''):`<div class="card" style="grid-column:1/-1">${empty('No open positions right now','New job openings will appear here when they are published.')}</div>`}</div>`;
    if($('addJob'))$('addJob').onclick=()=>formModal('Publish a job opening',field('title','Job title','text',true)+field('location','Location','text',true,'e.g. Bengaluru / Hybrid')+field('vacancies','Number of vacancies','number',true,'1','min="1"')+selectField('departmentId','Department',state.departments.map(d=>({value:d.id,label:d.name})))+field('description','Job description','text',true),'Publish job',v=>post('/jobs',{...v,vacancies:Number(v.vacancies),departmentId:v.departmentId?Number(v.departmentId):null}));
    $('pageBody').querySelectorAll('[data-apply]').forEach(b=>b.onclick=()=>applyJobModal(b.dataset.apply));$('pageBody').querySelectorAll('[data-job-edit]').forEach(b=>b.onclick=()=>editJob(b.dataset.jobEdit));
  }
  async function editJob(id){const j=state.jobs.find(x=>String(x.id)===String(id));if(!j)return;formModal('Edit job opening',field('title','Job title','text',true,'',`value="${esc(j.title)}"`)+field('location','Location','text',true,'',`value="${esc(j.location)}"`)+field('vacancies','Vacancies','number',true,'','min="1" value="'+(j.vacancies||1)+'"')+field('description','Description','text',true,'',`value="${esc(j.description)}"`)+selectField('status','Status',[{value:'OPEN',label:'Open'},{value:'CLOSED',label:'Closed'},{value:'ON_HOLD',label:'On Hold'}]),'Save changes',v=>put('/jobs/'+encodeURIComponent(id),{...v,vacancies:Number(v.vacancies)}));}
  function applyJobModal(id){
    const j=state.jobs.find(x=>String(x.id)===String(id));
    formModal('Apply for '+esc(j?.title||'Position'),field('name','Full name','text',true,'Your full name')+field('email','Email address','email',true,'Your email')+field('phone','Phone number','tel',false,'e.g. +91 9876543210')+field('resume','Resume / Portfolio link','url',false,'https://...'),'Submit application',v=>post(`/jobs/${encodeURIComponent(id)}/apply`,v));
  }
  async function renderCandidates(){
    headingAction('<button class="btn btn-outline" id="refreshCandidates">\u21bb Refresh</button>');const [jobData,candidates]=await Promise.all([get('/jobs'),get('/candidates')]);const jobs=arr(jobData);$('pageBody').innerHTML=`<section class="card card-pad"><div class="toolbar"><select id="candidateJob"><option value="">All job openings</option>${jobs.map(j=>`<option value="${esc(j.id)}">${esc(j.title)}</option>`).join('')}</select><select id="candidateStatus"><option value="">All statuses</option>${['APPLIED','SHORTLISTED','INTERVIEWED','SELECTED','REJECTED'].map(s=>`<option>${s}</option>`).join('')}</select><button class="btn btn-secondary btn-small" id="filterCandidates">Apply filters</button></div><div id="candidateTable">${candidateTable(arr(candidates))}</div></section>`;$('filterCandidates').onclick=async()=>{try{const q=new URLSearchParams();if($('candidateJob').value)q.set('jobId',$('candidateJob').value);if($('candidateStatus').value)q.set('status',$('candidateStatus').value);const d=await get('/candidates'+(q.size?'?'+q:''));$('candidateTable').innerHTML=candidateTable(arr(d));wireCandidateActions();}catch(e){toast(e.message,true);}};$('refreshCandidates').onclick=()=>navigate('candidates');wireCandidateActions();
  }
  function candidateTable(data){const rows=data.map(c=>`<tr><td><div class="person-cell"><span class="mini-avatar">${esc((c.name||'C').charAt(0).toUpperCase())}</span><span><strong>${fmt(c.name)}</strong><small>${fmt(c.email)} \u00b7 ${fmt(c.phone)}</small></span></div></td><td>${fmt(c.jobTitle)}</td><td>${status(c.status)}</td><td>${fmt(c.appliedAt)}</td><td><select data-status-id="${esc(c.id)}"><option value="">Update status\u2026</option>${['SHORTLISTED','INTERVIEWED','SELECTED','REJECTED'].map(s=>`<option value="${s}">${s}</option>`).join('')}</select></td><td><button class="btn btn-primary btn-small" data-convert="${esc(c.id)}" ${String(c.status).toUpperCase()!=='SELECTED'?'disabled':''}>Convert to Employee</button></td></tr>`).join('');return table(['Candidate','Job','Status','Applied','Change status','Onboarding'],rows,'Candidate applications will appear here.');}
  function wireCandidateActions(){
    $('pageBody').querySelectorAll('[data-status-id]').forEach(s=>s.onchange=async()=>{if(!s.value)return;try{await put('/candidates/'+encodeURIComponent(s.dataset.statusId)+'/status',{status:s.value});toast('Candidate status updated to '+s.value);navigate('candidates');}catch(e){toast(e.message,true);}});
    $('pageBody').querySelectorAll('[data-convert]').forEach(b=>b.onclick=async()=>{
      const cid=b.dataset.convert;
      const deps=arr(await get('/departments'));
      formModal('Convert candidate to employee',field('designation','Designation','text',true,'Job Title / Designation')+selectField('departmentId','Department',deps.map(d=>({value:d.id,label:d.name})))+field('phone','Phone number','tel',false,'Employee phone'),'Confirm conversion',v=>post('/candidates/'+encodeURIComponent(cid)+'/convert',{...v,departmentId:Number(v.departmentId)}));
    });
  }
  async function renderApplications(){const d=arr(await get('/candidates/me/applications'));const rows=d.map(a=>`<tr><td><strong>${fmt(a.jobTitle)}</strong></td><td>${fmt(a.location)}</td><td>${status(a.status)}</td><td>${fmt(a.appliedAt)}</td></tr>`).join('');$('pageBody').innerHTML=card('Application history',table(['Position','Location','Application status','Applied on'],rows,'Your submitted applications will appear here.'))+'<div class="section-gap"><button class="btn btn-outline" id="browseJobs">Browse open positions \u2192</button></div>';$('browseJobs').onclick=()=>navigate('jobs');}
  async function renderInterviews(){
    const isHr=can('ADMIN','HR_MANAGER');headingAction(isHr?'<button class="btn btn-primary" id="scheduleInterview">\uff0b Schedule interview</button>':'');const d=arr(await get('/interviews'));const rows=d.map(i=>`<tr><td><strong>${fmt(i.candidateName)}</strong></td><td>${fmt(i.jobTitle)}</td><td>${fmt(i.interviewerName)}</td><td>${fmt(i.scheduledAt)}</td><td>${fmt(i.mode)}</td><td>${status(i.status)}</td><td><button class="btn btn-outline btn-small" data-feedback="${esc(i.id)}">Feedback</button></td></tr>`).join('');$('pageBody').innerHTML=card('Interview schedule',table(['Candidate','Position','Interviewer','Date & time','Mode','Status','Action'],rows,'Scheduled interviews will appear here.'));
```

```
0144     if($('scheduleInterview'))$('scheduleInterview').onclick=async()=>{try{const [candData,emps]=await Promise.all([get('/candidates'),loadEmployeesList()]);const candidates=arr(cand
Data);formModal('Schedule interview',selectField('candidateId','Candidate',candidates.map(c=>({value:c.id,label:`${c.name} (${c.jobTitle||'Candidate'})`})))+selectField('interviewerId','I
nterviewer',emps.map(e=>({value:e.id,label:`${e.name} - ${e.designation}`})))+field('scheduledAt','Date & Time','datetime-local',true)+field('mode','Interview mode','text',false,'e.g. Goo
gle Meet, Microsoft Teams, On-site')+field('meetingLink','Meeting link or room','text',false,'https://...'),'Schedule interview',v=>post('/interviews',{...v,candidateId:Number(v.candidate
Id),interviewerId:Number(v.interviewerId)}));}catch(e){toast(e.message,true);}};
0145     $('pageBody').querySelectorAll('[data-feedback]').forEach(b=>b.onclick=()=>formModal('Interview feedback',field('feedback','Evaluation & feedback comments','text',true)+selectFie
ld('status','Interview outcome',[{value:'INTERVIEWED',label:'Interviewed (In Progress)'},{value:'SELECTED',label:'Selected (Offer Ready)'},{value:'REJECTED',label:'Rejected'}]),'Submit fe
edback',v=>put('/interviews/'+encodeURIComponent(b.dataset.feedback)+'/feedback',v)));
0146   }
0147   async function renderOnboarding(){
0148     const isHr=can('ADMIN','HR_MANAGER');headingAction(isHr?'<button class="btn btn-primary" id="addTask">\uff0b Assign task</button>':'');let tasks=[];
0149     if(can('EMPLOYEE')){const me=await get('/employees/me');tasks=arr(await get('/onboarding/employee/'+encodeURIComponent(me.id)));}else{tasks=arr(await get('/onboarding'));}
0150     const rows=tasks.map(t=>`<tr>${isHr?`<td><strong>${fmt(t.employeeName)}</strong><br><small class="muted">${fmt(t.employeeCode)}</small></td>`:''}<td><strong>${fmt(t.title||t.task
Name)}</strong></td><td>${fmt(t.description)}</td><td>${fmt(t.dueDate)}</td><td>${status(t.status)}</td><td><button class="btn btn-outline btn-small" data-task="${esc(t.id)}">Update</butt
on></td></tr>`).join('');
0151     const headers=isHr?['Employee','Task title','Details','Due date','Status','Action']:['Task title','Details','Due date','Status','Action'];
0152     $('pageBody').innerHTML=card('Onboarding checklist',table(headers,rows,'Onboarding tasks will appear here.'));
0153     if($('addTask'))$('addTask').onclick=async()=>{const emps=await loadEmployeesList();formModal('Assign onboarding task',selectField('employeeId','Employee',emps.map(e=>({value:e.i
d,label:`${e.name} (${e.employeeCode})`})))+field('title','Task title','text',true,'e.g. Submit identity documents')+field('description','Task description','text',false,'Detailed instruct
ions for employee')+field('dueDate','Due date','date',true),'Assign task',v=>post('/onboarding/tasks',{...v,employeeId:Number(v.employeeId)}));};
0154     $('pageBody').querySelectorAll('[data-task]').forEach(b=>b.onclick=()=>formModal('Update onboarding task',selectField('status','Task status',[{value:'PENDING',label:'Pending'},{v
alue:'IN_PROGRESS',label:'In progress'},{value:'COMPLETED',label:'Completed'}]),'Save status',v=>put('/onboarding/tasks/'+encodeURIComponent(b.dataset.task),v)));
0155   }
0156   async function renderSalary(){
0157     const isHr=can('ADMIN','HR_MANAGER','MANAGER');
0158     headingAction(isHr?'<button class="btn btn-primary" id="addSalary">\uff0b Add / Edit salary structure</button>':'');
0159     if(isHr){
0160       const data=arr(await get('/salary-structures'));
0161       const rows=data.map(s=>`<tr><td><strong>${fmt(s.employeeCode)}</strong></td><td><strong>${fmt(s.employeeName)}</strong><br><small class="muted">${fmt(s.designation)} \u00b7 ${f
mt(s.departmentName)}</small></td><td>${money(s.basic)}</td><td>${money(s.hra)}</td><td>${money(s.otherAllowances)}</td><td>${money(s.deductions)}</td><td><strong>${money(s.grossPay)}</st
rong></td><td><strong style="color:#16845b">${money(s.netPay)}</strong></td><td><button class="btn btn-outline btn-small" data-edit-salary="${esc(s.employeeId)}">Edit</button> <button cla
ss="btn btn-danger btn-small" data-del-salary="${esc(s.id)}">Delete</button></td></tr>`).join('');
0162       $('pageBody').innerHTML=card('Employee compensation & salary structures',table(['Employee code','Employee','Basic pay','HRA','Allowances','Deductions','Gross salary','Net month
ly pay','Actions'],rows,'No salary structures configured yet. Click Add / Edit salary structure to configure compensation.'));
0163       if($('addSalary'))$('addSalary').onclick=()=>openSalaryModal();
0164       $('pageBody').querySelectorAll('[data-edit-salary]').forEach(b=>b.onclick=()=>openSalaryModal(b.dataset.editSalary));
0165       $('pageBody').querySelectorAll('[data-del-salary]').forEach(b=>b.onclick=async()=>{if(confirm('Delete this salary structure?'))try{await del('/salary-structures/'+encodeURIComp
onent(b.dataset.delSalary));toast('Salary structure deleted.');renderSalary();}catch(e){toast(e.message,true);}});
0166     }else{
0167       try{
0168         const s=await get('/salary-structures/me');
0169         $('pageBody').innerHTML=card('My compensation structure',`<div class="notice section-gap">Your defined compensation breakdown registered with HR.</div><div class="field full
section-gap"><div class="detail-row"><span>Employee name</span><strong>${fmt(s.employeeName)}</strong></div><div class="detail-row"><span>Designation</span><strong>${fmt(s.designation)}</
strong></div><div class="detail-row"><span>Basic pay</span><strong>${money(s.basic)}</strong></div><div class="detail-row"><span>House Rent Allowance (HRA)</span><strong>${money(s.hra)}</
strong></div><div class="detail-row"><span>Other allowances</span><strong>${money(s.otherAllowances)}</strong></div><div class="detail-row"><span>Deductions (PF / Taxes)</span><strong>${m
oney(s.deductions)}</strong></div><div class="detail-row"><span>Gross salary</span><strong>${money(s.grossPay)}</strong></div><div class="detail-row"><span>Net monthly pay</span><strong s
tyle="color:#16845b;font-size:16px">${money(s.netPay)}</strong></div></div>`);
0170       }catch(err){$('pageBody').innerHTML=card('My compensation structure',empty('Salary structure not found','Your compensation structure has not yet been configured by HR.'));}
0171     }
0172   }
0173   async function openSalaryModal(empId=null){
0174     const emps=await loadEmployeesList();let current=null;
0175     if(empId){try{current=await get('/salary-structures/'+encodeURIComponent(empId));}catch{}}
0176     formModal(current?'Edit salary structure':'Configure salary structure',
0177       selectField('employeeId','Employee',emps.map(e=>({value:e.id,label:`${e.name} (${e.employeeCode}) - ${e.designation}`})))+
0178       field('basic','Basic pay (monthly)','number',true,'e.g. 50000','min="0" step="0.01" value="'+(current?.basic||'')+'"')+
0179       field('hra','HRA (House Rent Allowance)','number',true,'e.g. 20000','min="0" step="0.01" value="'+(current?.hra||'')+'"')+
0180       field('otherAllowances','Other allowances','number',false,'e.g. 10000','min="0" step="0.01" value="'+(current?.otherAllowances||'')+'"')+
0181       field('deductions','Custom deductions (PF / insurance)','number',false,'Leave blank for auto 12% PF','min="0" step="0.01" value="'+(current?.deductions||'')+'"'),
0182       'Save structure',
```

```
      v=>post('/salary-structures',{employeeId:Number(v.employeeId),basic:Number(v.basic),hra:Number(v.hra),otherAllowances:Number(v.otherAllowances||0),deductions:v.deductions?Numbe
r(v.deductions):null})
    );
    if(empId)setTimeout(()=>{const sel=document.getElementById('f_employeeId');if(sel)sel.value=empId;},50);
  }
  async function renderPayroll(){
    const isHr=can('ADMIN','HR_MANAGER');headingAction(isHr?'<button class="btn btn-primary" id="runPayroll">\u25b6 Run payroll</button>':'');
    if(isHr){
      const data=arr(await get('/payroll'));
      const rows=data.map(p=>`<tr><td><strong>${fmt(p.employeeName)}</strong><br><small class="muted">${fmt(p.employeeCode)} \u00b7 ${fmt(p.designation)}</small></td><td>${fmt(p.mont
h)} / ${fmt(p.year)}</td><td>${money(p.grossPay)}</td><td>${money(p.totalDeductions)}</td><td><strong style="color:#16845b">${money(p.netPay)}</strong></td><td>${status(p.status||'Process
ed')}</td><td><button class="btn btn-outline btn-small" data-payslip="${esc(p.employeeId)}" data-month="${esc(p.month)}" data-year="${esc(p.year)}">Download PDF</button></td></tr>`).join(
'');
      $('pageBody').innerHTML=card('Payroll operations & history',table(['Employee','Pay period','Gross earnings','Deductions','Net disbursed pay','Status','Payslip'],rows,'No payrol
l records generated yet. Click Run payroll to execute monthly payroll for active employees.'));
      $('pageBody').querySelectorAll('[data-payslip]').forEach(b=>b.onclick=()=>downloadPayslip(b.dataset.payslip,b.dataset.month,b.dataset.year));
      if($('runPayroll'))$('runPayroll').onclick=()=>formModal('Run monthly payroll',field('month','Payroll month (1\u201312)','number',true,'e.g. 10','min="1" max="12" value="'+(new
 Date().getMonth()+1)+'"')+field('year','Payroll year','number',true,'YYYY','min="2000" max="2100" value="'+new Date().getFullYear()+'"')+field('defaultBasic','Fallback basic pay (optiona
l)','number',false,'Applies if employee has no salary structure') ,'Execute payroll',v=>post('/payroll/run',{month:Number(v.month),year:Number(v.year),defaultBasic:v.defaultBasic?Number(v
.defaultBasic):null}));
    }else{
      try{
        const me=await get('/employees/me');const d=arr(await get('/payroll/'+encodeURIComponent(me.id)));
        const rows=d.map(p=>`<tr><td>${fmt(p.month)} / ${fmt(p.year)}</td><td>${money(p.grossSalary??p.grossPay)}</td><td>${money(p.totalDeductions)}</td><td><strong style="color:#16
845b">${money(p.netPay)}</strong></td><td>${status(p.status||'Processed')}</td><td><button class="btn btn-outline btn-small" data-payslip="${esc(me.id)}" data-month="${esc(p.month)}" data
-year="${esc(p.year)}">Download PDF</button></td></tr>`).join('');
        $('pageBody').innerHTML=card('My payslips & payment history',table(['Period','Gross pay','Total deductions','Net pay','Status','Payslip'],rows,'No payroll records found for y
our account.'));
        $('pageBody').querySelectorAll('[data-payslip]').forEach(b=>b.onclick=()=>downloadPayslip(b.dataset.payslip,b.dataset.month,b.dataset.year));
      }catch(e){$('pageBody').innerHTML=card('My payslips',`<div class="error-panel">${esc(e.message)}</div>`);}
    }
  }
  async function downloadPayslip(employeeId,month,year){try{const blob=await api(`/payroll/${encodeURIComponent(employeeId)}/payslip?month=${encodeURIComponent(month)}&year=${encodeU
RIComponent(year)}`);const url=URL.createObjectURL(blob);const a=document.createElement('a');a.href=url;a.download=`HRGenius-payslip-${month}-${year}.pdf`;a.click();URL.revokeObjectURL(ur
l);toast('Payslip downloaded successfully.');}catch(e){toast(e.message,true);}}
  async function renderPerformance(){
    const isHrOrMgr=can('ADMIN','HR_MANAGER','MANAGER');
    headingAction(isHrOrMgr?'<button class="btn btn-primary" id="addCycle">\uff0b Create cycle</button> <button class="btn btn-secondary" id="addGoal">\uff0b Assign goal</button> <bu
tton class="btn btn-outline" id="addReview">\uff0b Review</button>':'');
    const cycles=arr(await get('/performance/cycles'));
    const cycleRows=cycles.map(c=>`<tr><td><strong>${fmt(c.name)}</strong></td><td>${fmt(c.startDate)}</td><td>${fmt(c.endDate)}</td><td>${status(c.status||'Active')}</td></tr>`).joi
n('');
    let reviewRows='';
    if(isHrOrMgr){
      const reviews=arr(await get('/performance/reviews'));
      reviewRows=reviews.map(r=>`<tr><td><strong>${fmt(r.employeeName)}</strong></td><td>${fmt(r.cycleName)}</td><td><strong>${fmt(r.rating)} / 5</strong></td><td>${status(r.status)}
</td><td>${fmt(r.comments||r.selfAssessment)}</td></tr>`).join('');
    }else{
      const myReviews=arr(await get('/performance/reviews/me'));
      reviewRows=myReviews.map(r=>`<tr><td>${fmt(r.cycleName)}</td><td><strong>${fmt(r.rating)} / 5</strong></td><td>${status(r.status)}</td><td>${fmt(r.comments||'\u2014')}</td></tr
>`).join('');
    }
    $('pageBody').innerHTML=`${card('Performance cycles',table(['Cycle','Start date','End date','Status'],cycleRows,'Performance cycles will appear here.'))}<div class="dashboard-bot
tom section-gap">${card('Employee goals',`<div class="notice">${isHrOrMgr?'Managers and HR can assign strategic employee goals.':'Track your quarterly key results and goals.'}</div>${isHr
OrMgr?'<button class="btn btn-secondary btn-small section-gap" id="addGoalBody">\uff0b Assign goal</button>':''}`)}${card('Self assessment',`<div class="notice">Submit your reflections ag
ainst the active performance cycle.</div>${can('EMPLOYEE')?'<button class="btn btn-primary btn-small section-gap" id="selfAssess">Submit self-assessment</button>':''}`)}</div><div class="
section-gap">${card('Performance reviews',table(isHrOrMgr?['Employee','Cycle','Rating','Status','Comments / Self-reflection']:['Cycle','Rating','Status','Manager comments'],reviewRows,'Pe
rformance review records will appear here.'))}</div>`;
    const openGoalModal=async()=>{const emps=await loadEmployeesList();formModal('Assign employee goal',selectField('employeeId','Employee',emps.map(e=>({value:e.id,label:`${e.name}
(${e.employeeCode})`})))+field('title','Goal title','text',true,'e.g. Implement automated testing pipeline')+field('description','Goal description & KPIs','text',false)+field('dueDate','T
arget completion date','date',true),'Assign goal',v=>post('/performance/goals',{...v,employeeId:Number(v.employeeId)}));};
```

```
0220     if($('addCycle'))$('addCycle').onclick=()=>formModal('Create performance cycle',field('name','Cycle name','text',true,'e.g. Q4 2026 Review Cycle')+field('startDate','Start date',
'date',true)+field('endDate','End date','date',true),'Create cycle',v=>post('/performance/cycles',v));
0221     if($('addGoal'))$('addGoal').onclick=openGoalModal;if($('addGoalBody'))$('addGoalBody').onclick=openGoalModal;
0222     if($('addReview'))$('addReview').onclick=async()=>{const [emps,cyc]=await Promise.all([loadEmployeesList(),get('/performance/cycles')]);formModal('Create performance review',sele
ctField('employeeId','Employee',emps.map(e=>({value:e.id,label:`${e.name} (${e.employeeCode})`})))+selectField('cycleId','Performance cycle',arr(cyc).map(c=>({value:c.id,label:c.name})))+
field('rating','Rating (1\u20135)','number',true,'1 to 5','min="1" max="5"')+field('comments','Manager evaluation comments','text',true),'Save review',v=>post('/performance/reviews',{...v
,employeeId:Number(v.employeeId),cycleId:Number(v.cycleId),rating:Number(v.rating)}));};
0223     if($('selfAssess'))$('selfAssess').onclick=async()=>{const cyc=arr(await get('/performance/cycles'));formModal('Submit self assessment',selectField('cycleId','Performance cycle',
cyc.map(c=>({value:c.id,label:c.name})))+field('selfAssessment','Your self-reflection & key achievements','text',true),'Submit reflection',v=>post('/performance/self-assessments',{...v,cy
cleId:Number(v.cycleId)}));};
0224   }
0225   async function renderAnalytics(){
0226     const specs=[['Headcount','/analytics/headcount','bar'],['Hiring','/analytics/hiring','line'],['Attrition','/analytics/attrition','line'],['Payroll cost','/analytics/payroll-cost
','bar'],['Performance','/analytics/performance','bar']];const data=await Promise.all(specs.map(async s=>{try{return await get(s[1]);}catch(e){return {error:e.message};}}));state.charts.f
orEach(c=>c.destroy());state.charts=[];$('pageBody').innerHTML=`<div class="metric-grid">${specs.map((s,i)=>`<section class="card chart-card"><div class="card-title"><div><h3>${esc(s[0])}
</h3><p>Live calculated workforce data</p></div><span class="eyebrow eyebrow-dark">${String(i+1).padStart(2,'0')}</span></div>${data[i]?.error?`<div class="chart-placeholder">${esc(data[i
].error)}</div>`:`<canvas id="chart_${i}" aria-label="${esc(s[0])} chart"></canvas><div id="chartEmpty_${i}" class="chart-placeholder" hidden>No chartable data returned.</div>`}</section>
`).join('')}</div>`;
0227     specs.forEach((s,i)=>{if(data[i]?.error)return;const normalized=chartData(data[i]);const canvas=$('chart_'+i);if(!normalized.labels.length||!window.Chart){canvas.hidden=true;$('c
hartEmpty_'+i).hidden=false;return;}state.charts.push(new Chart(canvas,{type:s[2],data:{labels:normalized.labels,datasets:[{label:s[0],data:normalized.values,borderWidth:2,borderRadius:5,
tension:.35,backgroundColor:'rgba(40,117,199,.18)',borderColor:'#2875c7',fill:s[2]==='line'}]},options:{responsive:true,maintainAspectRatio:false,plugins:{legend:{display:false}},scales:{
y:{beginAtZero:true,grid:{color:'#edf1f5'},ticks:{font:{size:9}}},x:{grid:{display:false},ticks:{font:{size:9}}}}}}));});
0228   }
0229   function chartData(data){if(Array.isArray(data)){return {labels:data.map((x,i)=>String(x.label??x.month??x.name??x.period??i+1)),values:data.map(x=>Number(x.value??x.count??x.total
??x.amount??0))};}if(data&&Array.isArray(data.data))return chartData(data.data);if(data&&Array.isArray(data.labels)&&Array.isArray(data.values))return {labels:data.labels.map(String),valu
es:data.values.map(Number)};if(data&&typeof data==='object'){const entries=Object.entries(data).filter(([k,v])=>typeof v==='number');return {labels:entries.map(x=>x[0]),values:entries.map
(x=>x[1])};}return {labels:[],values:[]};}
0230   async function renderPassword(){$('pageBody').innerHTML=card('Update your password',`<form id="passwordForm" class="form-grid"><div class="field full"><label>Current password *</la
bel><input type="password" name="currentPassword" required autocomplete="current-password"></div><div class="field"><label>New password *</label><input type="password" name="newPassword"
minlength="8" required autocomplete="new-password"></div><div class="field"><label>Confirm new password *</label><input type="password" name="confirmPassword" minlength="8" required autoc
omplete="new-password"></div><div class="field full"><p class="muted">Use at least 8 characters and avoid reusing a previous password.</p></div><div class="field full"><p class="form-erro
r" id="passwordError"></p></div><div class="field full"><div class="form-actions"><button class="btn btn-primary" type="submit">Update password</button></div></div></form>`);$('passwordFo
rm').onsubmit=async e=>{e.preventDefault();const v=Object.fromEntries(new FormData(e.target));if(v.newPassword!==v.confirmPassword){$('passwordError').textContent='New password and confir
mation do not match.';return;}delete v.confirmPassword;try{await post('/auth/change-password',v);toast('Password changed successfully.');e.target.reset();}catch(err){$('passwordError').te
xtContent=err.message;}};
0231   }
0232   $('loginForm').onsubmit=async e=>{e.preventDefault();$('loginError').textContent='';const v=Object.fromEntries(new FormData(e.target));const b=e.target.querySelector('button');b.di
sabled=true;try{const d=await api('/auth/login',{method:'POST',body:JSON.stringify(v)});state.token=d.token||d.accessToken;state.role=d.role||d.roles?.[0];state.username=d.username||d.ema
il||v.username;if(!state.token||!state.role)throw new Error('The login response did not include a token and role.');showApp(state.username);}catch(err){$('loginError').textContent=err.mes
sage;}finally{b.disabled=false;}};
0233   $('candidateRegisterForm').onsubmit=async e=>{e.preventDefault();$('registerError').textContent='';const v=Object.fromEntries(new FormData(e.target));const b=e.target.querySelector
('button');b.disabled=true;try{await post('/auth/candidate-register',{...v,username:v.email});toast('Candidate account created. Sign in to continue.');showAuth('login');$('loginForm').ele
ments.username.value=v.email;}catch(err){$('registerError').textContent=err.message;}finally{b.disabled=false;}};
0234   document.querySelectorAll('[data-auth]').forEach(b=>b.onclick=()=>showAuth(b.dataset.auth));$('logoutBtn').onclick=()=>{state.token=null;state.role=null;state.username=null;state.c
harts.forEach(c=>c.destroy());state.charts=[];showAuth('login');};$('mobileMenu').onclick=()=>$('sidebar').classList.toggle('open');
0235   $('browsePublicJobs').onclick=async()=>{try{const jobs=arr(await api('/jobs'));const root=document.createElement('div');root.className='modal-backdrop';root.innerHTML=`<div class="
modal"><div class="modal-head"><h3>Public job openings</h3><button class="icon-button" data-close>\u00d7</button></div><div class="job-grid">${jobs.length?jobs.map(j=>`<article class="car
d job-card"><span class="eyebrow eyebrow-dark">${fmt(j.departmentName||'OPEN POSITION')}</span><h3>${fmt(j.title)}</h3><div class="job-meta">\u2316 ${fmt(j.location)} \u00b7 \u25a6 ${fmt(
j.vacancies)} vacancies</div><p>${fmt(j.description)}</p><button class="btn btn-primary btn-small" data-register-job>Sign in to apply</button></article>`).join(''):empty('No open position
s','New job openings will appear here when published.')}</div></div>`;document.body.append(root);root.querySelector('[data-close]').onclick=()=>root.remove();root.querySelectorAll('[data-
register-job]').forEach(b=>b.onclick=()=>{root.remove();showAuth('register');});}catch(e){toast(e.message,true);}};
0236   showAuth('login');
0237 })();
```

src/main/resources/static/index.html

```
0001 <!doctype html>
0002 <html lang="en">
0003 <head>
0004   <meta charset="utf-8">
0005   <meta name="viewport" content="width=device-width, initial-scale=1">
0006   <meta name="theme-color" content="#102b4e">
```

```
  <title>HRGenius | Human Resource Management</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Manrope:wght@500;600;700;800&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="/styles.css">
</head>
<body>
  <div id="toastRegion" class="toast-region" aria-live="polite"></div>
  <section id="authScreen" class="auth-screen">
    <div class="auth-art">
      <div class="brand brand-light"><span class="brand-mark">H</span><span>HR<span class="brand-accent">Genius</span></span></div>
      <div class="auth-art-copy"><span class="eyebrow">PEOPLE • PROCESS • PERFORMANCE</span><h1>Make work<br>work better.</h1><p>A connected workspace for your people operations, fro
m first application to career growth.</p></div>
      <div class="auth-art-footer"><span>HR Management System</span><span>01 / 06</span></div>
    </div>
    <div class="auth-panel-wrap">
      <div class="auth-panel">
        <div class="auth-mobile-brand brand"><span class="brand-mark">H</span><span>HR<span class="brand-accent">Genius</span></span></div>
        <div id="loginPane">
          <span class="eyebrow">WELCOME BACK</span><h2>Sign in to your workspace</h2><p class="muted">Enter your account details to continue.</p>
          <form id="loginForm" class="stack-form">
            <label>Email or username<input name="username" autocomplete="username" placeholder="you@company.com" required></label>
            <label>Password<input name="password" type="password" autocomplete="current-password" placeholder="Enter your password" required></label>
            <button class="btn btn-primary btn-wide" type="submit">Sign in <span>→</span></button>
            <p class="form-error" id="loginError" role="alert"></p>
          </form>
          <p class="auth-switch">Applying for a role? <button class="text-button" type="button" data-auth="register">Create a candidate account</button></p><p class="auth-switch"><bu
tton class="text-button" type="button" id="browsePublicJobs">Browse public job openings</button></p>
        </div>
        <div id="registerPane" hidden>
          <span class="eyebrow">JOIN OUR TALENT COMMUNITY</span><h2>Create candidate account</h2><p class="muted">Register to apply for open positions and track your applications.</p
>
          <form id="candidateRegisterForm" class="stack-form">
            <label>Full name<input name="name" autocomplete="name" required maxlength="120"></label>
            <label>Email address<input name="email" type="email" autocomplete="email" required></label>
            <label>Phone<input name="phone" autocomplete="tel" required></label>
            <label>Password<input name="password" type="password" autocomplete="new-password" minlength="8" required></label>
            <button class="btn btn-primary btn-wide" type="submit">Create account <span>→</span></button>
            <p class="form-error" id="registerError" role="alert"></p>
          </form>
          <p class="auth-switch">Already registered? <button class="text-button" type="button" data-auth="login">Sign in</button></p>
        </div>
      </div>
      <div class="auth-legal">© HRGenius · Secure people operations</div>
    </div>
  </section>

  <div id="appShell" class="app-shell" hidden>
    <aside class="sidebar" id="sidebar">
      <div class="brand sidebar-brand"><span class="brand-mark">H</span><span>HR<span class="brand-accent">Genius</span></span></div>
      <div class="workspace-label">WORKSPACE</div>
      <nav id="sideNav" class="side-nav" aria-label="Main navigation"></nav>
      <div class="sidebar-bottom"><div class="help-card"><span class="help-icon">✦</span><strong>Need a hand?</strong><small>Contact your HR administrator for access and support.</s
mall></div><button id="logoutBtn" class="logout-button"><span>■</span> Sign out</button></div>
    </aside>
    <main class="main-area">
      <header class="topbar"><button id="mobileMenu" class="icon-button mobile-menu" aria-label="Toggle menu">■</button><div class="breadcrumb"><span>Workspace</span><b>/</b><strong
 id="pageCrumb">Overview</strong></div><div class="topbar-right"><span class="topbar-date" id="todayLabel"></span><div class="user-chip"><span class="avatar" id="userAvatar">U</span><span
><strong id="userName">User</strong><small id="userRole">ROLE</small></span></div></div></header>
      <div class="page-wrap"><div id="pageContent" aria-live="polite"></div></div>
      <footer class="app-footer"><span>HRGenius HRMS</span><span>People operations, made clear.</span></footer>
```

```
0062     </main>
0063   </div>
0064   <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.4/dist/chart.umd.min.js" defer></script>
0065   <script src="/app.js" defer></script>
0066 </body>
0067 </html>
```

src/main/resources/static/styles.css

```
0001 [hidden],.hidden{
0002   display:none!important}
0003 }
0004 :root{
0005   --navy:#102b4e;
0006   --navy2:#173c66;
0007   --blue:#2875c7;
0008   --blue2:#eaf3fd;
0009   --ink:#172536;
0010   --muted:#718096;
0011   --line:#e7edf3;
0012   --canvas:#f5f7fa;
0013   --white:#fff;
0014   --green:#16845b;
0015   --amber:#b7791f;
0016   --red:#c2414b;
0017   --shadow:0 8px 28px rgba(20,43,72,.06)}
0018 }
0019 *{
0020   box-sizing:border-box}
0021 }
0022 body{
0023   margin:0;
0024   background:var(--canvas);
0025   color:var(--ink);
0026   font:14px 'DM Sans',sans-serif}
0027 }
0028 button,input,select,textarea{
0029   font:inherit}
0030 }
0031 button{
0032   cursor:pointer}
0033 }
0034 h1,h2,h3,h4{
0035   font-family:Manrope,'DM Sans',sans-serif;
0036   letter-spacing:-.035em;
0037   margin:0}
0038 }
0039 p{
0040   line-height:1.6}
0041 }
0042 .muted{
0043   color:var(--muted)}
0044 }
0045 .eyebrow{
0046   font-size:10px;
0047   letter-spacing:.17em;
0048   font-weight:700;
0049   color:#6993bf}
0050 }
0051 .brand{
0052   display:flex;
0053   align-items:center;
```

```
  gap:10px;
  font:800 21px Manrope,sans-serif;
  color:var(--navy);
  letter-spacing:-.06em}
}
.brand-mark{
  width:32px;
  height:32px;
  display:grid;
  place-items:center;
  background:#2875c7;
  color:white;
  border-radius:10px;
  font:800 19px Manrope;
  box-shadow:0 5px 12px #2875c733}
}
.brand-accent{
  color:#2875c7}
}
.brand-light{
  color:white}
}
.brand-light .brand-accent{
  color:#83baff}
}
.auth-screen{
  min-height:100vh;
  display:grid;
  grid-template-columns:minmax(360px, .92fr) 1.08fr;
  background:#fff}
}
.auth-art{
  padding:44px 8%;
  background:radial-gradient(circle at 82% 18%,#24588c 0,transparent 34%),linear-gradient(145deg,#0e2949,#153b63 68%,#0c2441);
  color:#fff;
  display:flex;
  flex-direction:column;
  justify-content:space-between;
  position:relative;
  overflow:hidden}
}
.auth-art:after{
  content:"";
  position:absolute;
  width:520px;
  height:520px;
  border:1px solid #ffffff18;
  border-radius:50%;
  right:-210px;
  bottom:-270px;
  box-shadow:0 0 0 55px #ffffff08,0 0 0 110px #ffffff06}
}
.auth-art-copy{
  max-width:490px;
  position:relative;
  z-index:1;
  margin-top:-20px}
}
.auth-art-copy h1{
  font-size:clamp(48px,6vw,76px);
  line-height:1.04;
```

```
  margin:20px 0}
}
.auth-art-copy p{
  max-width:390px;
  color:#c0d0e1;
  font-size:16px}
}
.auth-art-footer{
  display:flex;
  justify-content:space-between;
  color:#9eb4cb;
  font-size:11px;
  letter-spacing:.1em;
  text-transform:uppercase}
}
.auth-panel-wrap{
  display:flex;
  flex-direction:column;
  align-items:center;
  justify-content:center;
  padding:40px}
}
.auth-panel{
  width:min(430px,100%)}
}
.auth-panel h2{
  font-size:30px;
  margin:12px 0 8px}
}
.auth-panel>.muted{
  margin:0 0 28px}
}
.auth-mobile-brand{
  display:none;
  margin-bottom:35px}
}
.stack-form{
  display:grid;
  gap:17px}
}
.stack-form label,.field label{
  display:grid;
  gap:8px;
  font-size:12px;
  font-weight:700;
  color:#405268}
}
.stack-form input,.field input,.field select,.field textarea,.toolbar input,.toolbar select{
  width:100%;
  border:1px solid #dce4ec;
  background:#fff;
  border-radius:8px;
  padding:12px 13px;
  outline:none;
  color:var(--ink);
  transition:.2s}
}
.stack-form input:focus,.field input:focus,.field select:focus,.field textarea:focus,.toolbar input:focus{
  border-color:#6da5df;
  box-shadow:0 0 0 3px #2875c71a}
}
```

```
.btn{
  border:0;
  border-radius:8px;
  padding:11px 16px;
  font-weight:700;
  display:inline-flex;
  align-items:center;
  justify-content:center;
  gap:12px;
  transition:.18s}
}
.btn-primary{
  background:var(--blue);
  color:white;
  box-shadow:0 5px 12px #2875c729}
}
.btn-primary:hover{
  background:#1d65b0;
  transform:translateY(-1px)}
}
.btn-secondary{
  background:#edf3f9;
  color:#28547e}
}
.btn-outline{
  background:white;
  color:#42566c;
  border:1px solid #dce4ec}
}
.btn-danger{
  background:#fff0f0;
  color:#b93843}
}
.btn-small{
  padding:7px 10px;
  font-size:12px}
}
.btn-wide{
  width:100%;
  margin-top:5px;
  padding:14px}
}
.text-button{
  border:0;
  background:none;
  padding:0;
  color:#2875c7;
  font-weight:700}
}
.auth-switch{
  margin-top:25px;
  text-align:center;
  color:var(--muted);
  font-size:12px}
}
.auth-legal{
  font-size:11px;
  color:#a0aab6;
  margin-top:48px}
}
.form-error{
```

```
  color:var(--red);
  font-size:12px;
  min-height:16px;
  margin:0}
}
.app-shell{
  min-height:100vh}
}
.sidebar{
  width:248px;
  background:#102b4e;
  color:#d5e0eb;
  position:fixed;
  inset:0 auto 0 0;
  display:flex;
  flex-direction:column;
  padding:25px 16px 18px;
  z-index:20}
}
.sidebar-brand{
  color:white;
  padding:0 10px 38px}
}
.sidebar-brand .brand-accent{
  color:#8bbcff}
}
.workspace-label{
  font-size:9px;
  letter-spacing:.18em;
  color:#7891aa;
  font-weight:700;
  padding:0 12px 12px}
}
.side-nav{
  display:grid;
  gap:5px}
}
.nav-item{
  width:100%;
  display:flex;
  align-items:center;
  gap:12px;
  border:0;
  background:transparent;
  color:#b7c7d8;
  text-align:left;
  padding:11px 13px;
  border-radius:8px;
  font-weight:600;
  font-size:13px}
}
.nav-item:hover{
  background:#ffffff0d;
  color:white}
}
.nav-item.active{
  background:#2875c7;
  color:white;
  box-shadow:0 5px 14px #071d3440}
}
.nav-icon{
```

```
  width:19px;
  text-align:center;
  font-size:16px}
}
.sidebar-bottom{
  margin-top:auto}
}
.help-card{
  background:#183b61;
  border:1px solid #315577;
  border-radius:10px;
  padding:14px;
  margin:0 3px 17px;
  display:grid;
  gap:6px}
}
.help-icon{
  color:#83baff;
  font-size:18px}
}
.help-card strong{
  font-size:12px;
  color:#fff}
}
.help-card small{
  color:#a8bdd1;
  line-height:1.5;
  font-size:10px}
}
.logout-button{
  width:100%;
  border:0;
  background:transparent;
  color:#b9c9d9;
  padding:12px;
  text-align:left;
  font-weight:600}
}
.logout-button span{
  margin-right:10px}
}
.main-area{
  margin-left:248px;
  min-height:100vh;
  display:flex;
  flex-direction:column}
}
.topbar{
  height:72px;
  background:#fff;
  border-bottom:1px solid var(--line);
  display:flex;
  align-items:center;
  justify-content:space-between;
  padding:0 38px;
  position:sticky;
  top:0;
  z-index:10}
}
.breadcrumb{
  display:flex;
```

```
  align-items:center;
  gap:10px;
  font-size:12px;
  color:#98a5b3}
}
.breadcrumb b{
  font-weight:400;
  color:#d0d8e0}
}
.breadcrumb strong{
  color:#263a50}
}
.topbar-right{
  display:flex;
  align-items:center;
  gap:24px}
}
.topbar-date{
  font-size:11px;
  color:#8a98a8}
}
.user-chip{
  display:flex;
  align-items:center;
  gap:9px}
}
.user-chip>span:last-child{
  display:grid;
  gap:2px}
}
.user-chip strong{
  font-size:11px}
}
.user-chip small{
  font-size:9px;
  color:#8392a3;
  letter-spacing:.08em}
}
.avatar{
  width:34px;
  height:34px;
  border-radius:50%;
  display:grid;
  place-items:center;
  background:#e8f1fb;
  color:#2868a6;
  font-weight:800}
}
.page-wrap{
  width:min(1440px,100%);
  padding:32px 38px 45px;
  flex:1}
}
.app-footer{
  padding:14px 38px;
  border-top:1px solid var(--line);
  display:flex;
  justify-content:space-between;
  color:#9aa6b3;
  font-size:10px}
}
```

```
.mobile-menu{
  display:none}
}
.page-heading{
  display:flex;
  justify-content:space-between;
  align-items:flex-start;
  gap:20px;
  margin-bottom:26px}
}
.page-heading h1{
  font-size:27px;
  margin:6px 0}
}
.page-heading p{
  margin:0;
  color:var(--muted);
  font-size:12px}
}
.heading-actions{
  display:flex;
  gap:8px;
  flex-wrap:wrap}
}
.eyebrow-dark{
  color:#6e8eaf}
}
.card{
  background:#fff;
  border:1px solid #edf1f5;
  border-radius:11px;
  box-shadow:var(--shadow)}
}
.card-pad{
  padding:20px}
}
.card-title{
  display:flex;
  justify-content:space-between;
  align-items:center;
  gap:12px;
  margin-bottom:16px}
}
.card-title h3{
  font-size:14px}
}
.card-title p{
  font-size:11px;
  color:var(--muted);
  margin:4px 0 0}
}
.stats-grid{
  display:grid;
  grid-template-columns:repeat(4,minmax(0,1fr));
  gap:14px;
  margin-bottom:18px}
}
.stat-card{
  padding:18px 19px;
  position:relative;
  overflow:hidden}
```

```
}
.stat-label{
  font-size:11px;
  color:#8492a2;
  font-weight:600}
}
.stat-value{
  font:800 27px Manrope,sans-serif;
  margin:11px 0 5px;
  color:#20364f}
}
.stat-note{
  font-size:10px;
  color:#8c9aaa}
}
.stat-icon{
  position:absolute;
  right:17px;
  top:16px;
  width:34px;
  height:34px;
  display:grid;
  place-items:center;
  border-radius:9px;
  background:#edf5fd;
  color:#2875c7;
  font-size:16px}
}
.dashboard-grid{
  display:grid;
  grid-template-columns:1.35fr 1fr;
  gap:16px}
}
.dashboard-bottom{
  display:grid;
  grid-template-columns:1fr 1fr;
  gap:16px;
  margin-top:16px}
}
.empty-state{
  padding:35px 18px;
  text-align:center;
  color:#8a98a8}
}
.empty-state .empty-icon{
  font-size:26px;
  color:#a8bfd7;
  margin-bottom:9px}
}
.empty-state strong{
  display:block;
  color:#53677d;
  font-size:12px;
  margin-bottom:5px}
}
.empty-state p{
  font-size:11px;
  margin:0}
}
.toolbar{
  display:flex;
```

```
  gap:9px;
  align-items:center;
  flex-wrap:wrap;
  margin-bottom:15px}
}
.toolbar input{
  max-width:270px;
  padding:10px 12px;
  font-size:12px}
}
.toolbar select{
  width:auto;
  min-width:140px;
  padding:10px 12px;
  font-size:12px}
}
.table-wrap{
  overflow:auto}
}
table{
  width:100%;
  border-collapse:collapse;
  white-space:nowrap}
}
th{
  text-align:left;
  padding:11px 13px;
  color:#8492a2;
  font-size:9px;
  letter-spacing:.1em;
  text-transform:uppercase;
  background:#f9fbfd;
  border-bottom:1px solid var(--line)}
}
td{
  padding:13px;
  border-bottom:1px solid #eef2f6;
  font-size:11px;
  color:#415368}
}
tbody tr:last-child td{
  border-bottom:0}
}
.person-cell{
  display:flex;
  align-items:center;
  gap:9px}
}
.mini-avatar{
  width:28px;
  height:28px;
  display:grid;
  place-items:center;
  background:#edf4fb;
  color:#3673ad;
  border-radius:8px;
  font-size:10px;
  font-weight:700}
}
.person-cell strong{
  display:block;
```

```
  color:#263b51;
  font-size:11px}
}
.person-cell small{
  display:block;
  color:#91a0af;
  font-size:9px;
  margin-top:3px}
}
.status{
  display:inline-flex;
  align-items:center;
  gap:5px;
  border-radius:30px;
  padding:5px 9px;
  font-size:9px;
  font-weight:700;
  background:#eef3f7;
  color:#65788b}
}
.status:before{
  content:"";
  width:5px;
  height:5px;
  border-radius:50%;
  background:currentColor}
}
.status.active,.status.selected,.status.completed,.status.paid{
  background:#e8f7f0;
  color:#16845b}
}
.status.pending,.status.applied,.status.in-progress{
  background:#fff5e5;
  color:#b7791f}
}
.status.rejected,.status.inactive{
  background:#fff0f0;
  color:#c2414b}
}
.status.shortlisted,.status.interviewed,.status.open{
  background:#eaf3fd;
  color:#2875c7}
}
.pagination{
  display:flex;
  justify-content:space-between;
  align-items:center;
  padding:13px 14px;
  color:#8b98a7;
  font-size:10px;
  border-top:1px solid var(--line)}
}
.pagination div{
  display:flex;
  gap:5px}
}
.pagination button{
  border:1px solid #e2e9ef;
  background:white;
  border-radius:6px;
  padding:6px 9px;
```

```
  color:#52677d}
}
.pagination button:disabled{
  opacity:.4;
  cursor:not-allowed}
}
.form-grid{
  display:grid;
  grid-template-columns:repeat(2,minmax(0,1fr));
  gap:13px}
}
.field{
  display:grid;
  gap:7px}
}
.field.full{
  grid-column:1/-1}
}
.field label{
  font-size:11px}
}
.field input,.field select,.field textarea{
  font-size:12px;
  padding:10px 11px}
}
.field textarea{
  min-height:80px;
  resize:vertical}
}
.form-actions{
  display:flex;
  justify-content:flex-end;
  gap:8px;
  margin-top:16px}
}
.split-grid{
  display:grid;
  grid-template-columns:minmax(260px,.8fr) 1.4fr;
  gap:16px}
}
.section-gap{
  margin-top:16px}
}
.job-grid{
  display:grid;
  grid-template-columns:repeat(2,minmax(0,1fr));
  gap:13px}
}
.job-card{
  padding:17px}
}
.job-card h3{
  font-size:14px;
  margin:7px 0}
}
.job-meta{
  display:flex;
  gap:12px;
  color:#8392a2;
  font-size:10px;
  margin:9px 0}
```

```
}
.job-card p{
  font-size:11px;
  color:#6f7f90;
  margin:8px 0 14px}
}
.job-card-footer{
  display:flex;
  justify-content:space-between;
  align-items:center}
}
.metric-grid{
  display:grid;
  grid-template-columns:repeat(2,minmax(0,1fr));
  gap:15px}
}
.chart-card{
  padding:18px;
  min-height:280px}
}
.chart-card canvas{
  width:100%!important;
  max-height:215px}
}
.chart-placeholder{
  height:205px;
  display:grid;
  place-items:center;
  color:#9aa7b4;
  font-size:11px}
}
.detail-row{
  display:flex;
  justify-content:space-between;
  padding:10px 0;
  border-bottom:1px solid #edf1f5;
  font-size:11px}
}
.detail-row:last-child{
  border:0}
}
.notice{
  padding:12px 14px;
  background:#eef6ff;
  border:1px solid #d8e9fa;
  color:#46698c;
  border-radius:8px;
  font-size:11px;
  line-height:1.6}
}
.error-panel{
  padding:13px 15px;
  background:#fff2f2;
  border:1px solid #f6d5d5;
  color:#a83a42;
  border-radius:8px;
  font-size:11px}
}
.modal-backdrop{
  position:fixed;
  inset:0;
```

```
  background:#0d213b80;
  z-index:50;
  display:grid;
  place-items:center;
  padding:18px}
}
.modal{
  width:min(620px,100%);
  max-height:90vh;
  overflow:auto;
  background:white;
  border-radius:13px;
  padding:23px;
  box-shadow:0 20px 70px #071b3340}
}
.modal-head{
  display:flex;
  justify-content:space-between;
  align-items:center;
  margin-bottom:18px}
}
.modal-head h3{
  font-size:17px}
}
.icon-button{
  border:0;
  background:#edf3f8;
  color:#50677e;
  border-radius:7px;
  width:32px;
  height:32px}
}
.toast-region{
  position:fixed;
  top:18px;
  right:18px;
  z-index:100;
  display:grid;
  gap:8px}
}
.toast{
  padding:12px 16px;
  background:#173c66;
  color:white;
  border-radius:8px;
  box-shadow:var(--shadow);
  font-size:12px;
  max-width:360px}
}
.toast.error{
  background:#a83a42}
}
.hidden{
  display:none!important}
}
@media(max-width:1100px){
.sidebar{
    width:220px}
  }
.main-area{
    margin-left:220px}
```

```
  }
.page-wrap{
    padding:28px 24px}
  }
.topbar{
    padding:0 24px}
  }
.stats-grid{
    grid-template-columns:repeat(2,1fr)}
  }
}
@media(max-width:760px){
.auth-screen{
    grid-template-columns:1fr}
  }
.auth-art{
    display:none}
  }
.auth-panel-wrap{
    min-height:100vh;
    padding:28px}
  }
.auth-mobile-brand{
    display:flex}
  }
.sidebar{
    transform:translateX(-100%);
    transition:.25s;
    width:255px}
  }
.sidebar.open{
    transform:translateX(0);
    box-shadow:10px 0 35px #102b4e33}
  }
.main-area{
    margin-left:0}
  }
.topbar{
    height:62px;
    padding:0 16px}
  }
.mobile-menu{
    display:block}
  }
.breadcrumb{
    margin-right:auto;
    margin-left:10px}
  }
.topbar-date{
    display:none}
  }
.page-wrap{
    padding:23px 15px 32px}
  }
.page-heading{
    display:block}
  }
.heading-actions{
    margin-top:15px}
  }
.stats-grid{
```

```
    grid-template-columns:repeat(2,minmax(0,1fr));
    gap:9px}
  }
.stat-card{
    padding:14px}
  }
.stat-value{
    font-size:22px}
  }
.dashboard-grid,.dashboard-bottom,.split-grid,.metric-grid{
    grid-template-columns:1fr}
  }
.job-grid{
    grid-template-columns:1fr}
  }
.app-footer{
    padding:13px 16px}
  }
.form-grid{
    grid-template-columns:1fr}
  }
.field.full{
    grid-column:auto}
  }
.user-chip>span:last-child{
    display:none}
  }
}
@media(max-width:420px){
.stats-grid{
    grid-template-columns:1fr 1fr}
  }
.stat-icon{
    display:none}
  }
.auth-panel h2{
    font-size:26px}
  }
}
```

03. BUILD & CONFIGURATION FILES

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion><groupId>com.hrgenius</groupId><artifactId>hrgenius</artifactId><version>1.0.0</version>
  <parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-parent</artifactId><version>3.3.5</version><relativePath/></parent>
  <properties><java.version>17</java.version><jjwt.version>0.12.6</jjwt.version></properties>
  <dependencies>
   <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId></dependency>
   <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
   <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-security</artifactId></dependency>
   <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-validation</artifactId></dependency>
   <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>${jjwt.version}</version></dependency>
   <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId><version>${jjwt.version}</version><scope>runtime</scope></dependency>
   <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId><version>${jjwt.version}</version><scope>runtime</scope></dependency>
   <dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-j</artifactId><scope>runtime</scope></dependency>
   <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><scope>runtime</scope></dependency>
   <dependency><groupId>org.springdoc</groupId><artifactId>springdoc-openapi-starter-webmvc-ui</artifactId><version>2.6.0</version></dependency>
   <dependency><groupId>com.github.librepdf</groupId><artifactId>openpdf</artifactId><version>2.0.3</version></dependency>
   <dependency><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId><optional>true</optional></dependency>
   <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope></dependency>
   <dependency><groupId>org.springframework.security</groupId><artifactId>spring-security-test</artifactId><scope>test</scope></dependency>
  </dependencies>
  <build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId></plugin></plugins></build>
</project>
```

postman/HRGenius.postman_collection.json

```
{
  "info": {
    "name": "HRGenius API Starter",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "variable": [
    {
      "key": "baseUrl",
      "value": "http://localhost:8080/api/v1"
    },
    {
      "key": "token",
      "value": ""
    }
  ],
  "item": [
    {
      "name": "Login",
      "request": {
        "method": "POST",
        "header": [
          {
            "key": "Content-Type",
            "value": "application/json"
          }
        ],
        "body": {
          "mode": "raw",
          "raw": "{\"username\":\"admin\",\"password\":\"Admin@12345\"}"
        },
```

```
        "url": "{{baseUrl}}/auth/login"
      }
    },
    {
      "name": "Create Employee",
      "request": {
        "method": "POST",
        "header": [
          {
            "key": "Content-Type",
            "value": "application/json"
          },
          {
            "key": "Authorization",
            "value": "Bearer {{token}}"
          }
        ],
        "body": {
          "mode": "raw",
          "raw": "{\"name\":\"Anita Sharma\",\"email\":\"anita.sharma@example.com\",\"phone\":\"9876543210\",\"designation\":\"Software Engineer\",\"departmentId\":1,\"dateOfJoining\
":\"2026-08-01\"}"
        },
        "url": "{{baseUrl}}/employees"
      }
    },
    {
      "name": "List Employees",
      "request": {
        "method": "GET",
        "header": [
          {
            "key": "Authorization",
            "value": "Bearer {{token}}"
          }
        ],
        "url": "{{baseUrl}}/employees?page=0&size=20&search=Anita"
      }
    },
    {
      "name": "List Departments",
      "request": {
        "method": "GET",
        "header": [
          {
            "key": "Authorization",
            "value": "Bearer {{token}}"
          }
        ],
        "url": "{{baseUrl}}/departments"
      }
    }
  ]
}
```

src/main/resources/application-mysql.properties

```
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/hrgenius?createDatabaseIfNotExist=true}
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
```

src/main/resources/application-test.properties

```
spring.datasource.url=jdbc:h2:mem:hrgenius;MODE=MySQL;DB_CLOSE_DELAY=-1;NON_KEYWORDS=USER,MONTH,YEAR
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
app.jwt.secret=test-secret-key-that-is-at-least-thirty-two-bytes-long
```

src/main/resources/application.properties

```
spring.application.name=HRGenius
server.port=${PORT:8081}

# Database Configuration (Defaults to embedded H2 for instant out-of-the-box execution)
spring.datasource.url=${DB_URL:jdbc:h2:mem:hrgenius;MODE=MySQL;DB_CLOSE_DELAY=-1;NON_KEYWORDS=USER,MONTH,YEAR}
spring.datasource.driver-class-name=${DB_DRIVER:org.h2.Driver}
spring.datasource.username=${DB_USERNAME:sa}
spring.datasource.password=${DB_PASSWORD:}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.jpa.show-sql=false
spring.jackson.serialization.write-dates-as-timestamps=false

# JWT Configuration
app.jwt.secret=${JWT_SECRET:change-this-development-secret-key-to-at-least-32-bytes}
app.jwt.expiration-ms=${JWT_EXPIRATION_MS:86400000}
```